



二哥的Java进阶之路

javabetter.cn

面渣逆袭 集合框架篇

三分恶|沉默王二修订版

7000字38张手绘图

内含29道面试八股

已成功帮助 7000 名球友拿到心仪 offer



前言

14554 字 67 张手绘图，详解 29 道 Java 集合框架面试高频题（让天下没有难背的八股），面渣背会这些 Java 容器八股文，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接](#)，作者：三分恶，戳[原文链接](#)。

亮白版本更适合拿出来打印，这也是很多学生党喜欢的方式，打印出来背诵的效率会更高。

Map

Map 中最重要的就是 HashMap 了，面试基本被问出包浆了，一定要好好准备。

8.能说一下 HashMap 的底层数据结构吗？

推荐阅读：二哥的 Java 进阶之路：详解 HashMap

JDK 8 中 HashMap 的数据结构是 数组 + 链表 + 红黑树。

桶数组

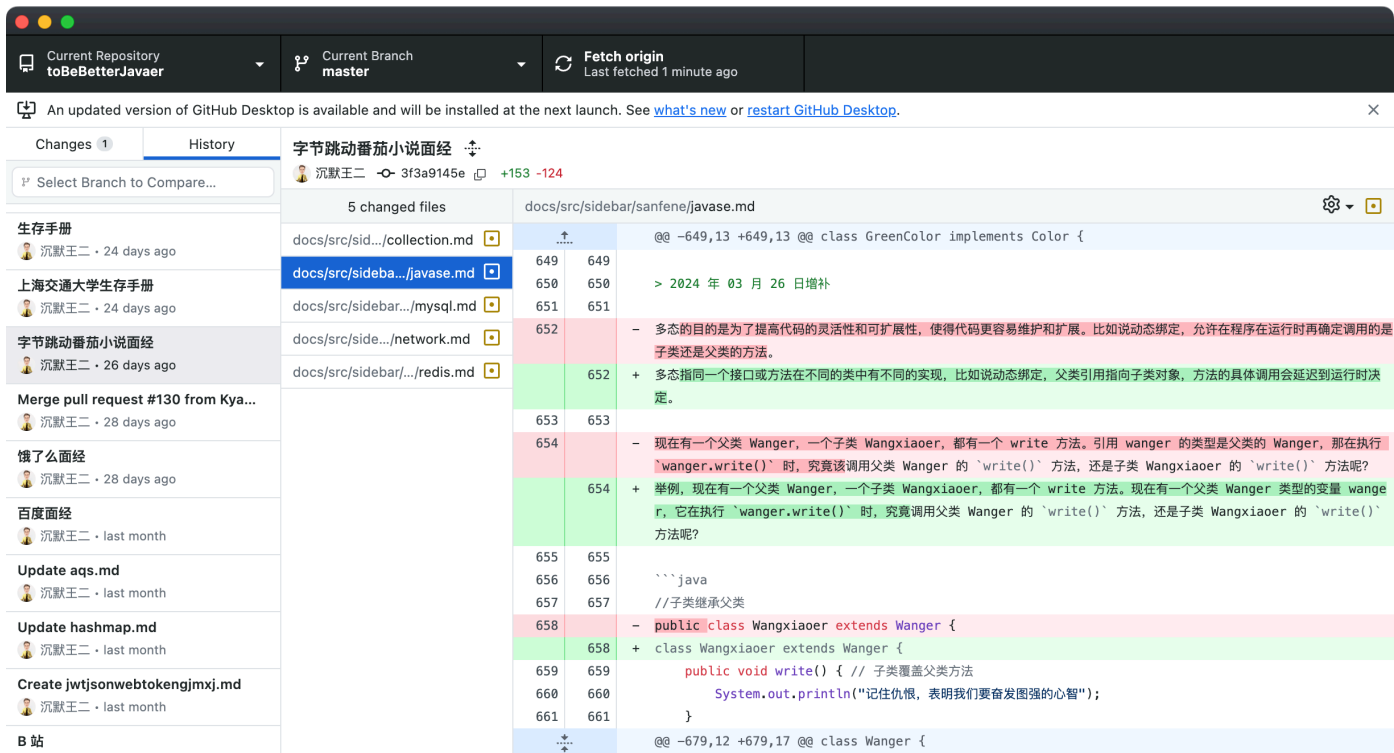
链表

红黑树

数组用来存储键值对，每个键值对可以通过索引直接拿到，索引是通过键的哈希值进行进一步的 `hash()` 处理得到的。

2024 年 12 月 30 日开始着手第二版更新。

- 对于高频题，会标注在《[Java 面试指南 \(付费\)](#)》中出现的位置，哪家公司，原题是什么，并且会加🌟，目录一目了然；如果你想节省时间的话，可以优先背诵这些题目，尽快做到知彼知己，百战不殆。
- 区分八股精华回答版本和原理底层解释，让大家知其然知其所以然，同时又能做到面试时的高效回答。
- 结合项目（[技术派](#)、[pmhub](#)）来组织语言，让面试官最大程度感受到你的诚意，而不是机械化的背诵。
- 修复第一版中出现的问题，包括球友们的私信反馈，网站留言区的评论，以及 [GitHub 仓库](#)中的 issue，让这份面试指南更加完善。
- 增加[二哥编程星球](#)的球友们拿到的一些 offer，对面渣逆袭的感谢，以及对简历修改的一些认可，以此来激励大家，给大家更多信心。
- 优化排版，增加手绘图，重新组织答案，使其更加口语化，从而更贴近面试官的预期。



由于 PDF 没办法自我更新，所以需要最新版的小伙伴，可以微信搜【沉默王二】，或者扫描/长按识别下面的二维码，关注二哥的公众号，回复【222】即可拉取最新版本。



当然了，请允许我的一点点私心，那就是星球的 PDF 版本会比公众号早一个月时间，毕竟星球用户都付费过了，我有必要让他们先享受到一点点福利。相信大家也都能理解，毕竟在线版是免费的，CDN、服务器、域名、OSS 等等都是需要成本的。

更别说我付出的时间和精力了，大家觉得有帮助还请给个口碑，让你身边的同事、同学都能受益到。

The image is a screenshot of a WeChat interface. On the left, there's a chat window with a contact named 'wanganpan'. The chat history shows a message about a salary of 60*13 in Xinjiang and an assignment to Iraq. Below that, a message from the contact '二哥' (Brother 2) is shown, mentioning a Java PDF, GitHub stars, and links to 'javabetter.cn'. On the right, a file list is displayed, showing various PDF and EPUB files related to Java, JVM, and system operations, with their respective upload dates and times. The files include '二哥的 JVM 进阶之路', '面渣逆袭 Java 基础篇', and '面渣逆袭 JVM 篇' in both EPUB and PDF formats, some with '暗黑版' (Dark Version) labels. The interface includes standard WeChat navigation icons at the bottom and top.

我把二哥的 Java 进阶之路、JVM 进阶之路、并发编程进阶之路，以及所有面渣逆袭的版本都放进来了，涵盖 Java 基础、Java 集合、Java 并发、JVM、Spring、MyBatis、计算机网络、操作系统、MySQL、Redis、RocketMQ、分布式、微服务、设计模式、Linux 等 16 个大的主题，共有 40 多万字，2000+张手绘图，可以说是诚意满满。

展示一下暗黑版本的 PDF 吧，排版清晰，字体优雅，更加适合夜服，晚上看会更舒服一点。

文件 主页 注释 视图 表单 保护 共享 帮助

开始

面渣逆袭集合框架篇V... 面渣逆袭集合框架篇...

31 32 33 34

Table[] 为空

是

数组添加元素

否

key是否相同

是

覆盖元素

否

是否树节点

是

红黑树插入元素

否

达到树化阈值

覆盖链表节点

链表树化

容量判断, 是否超过阈值

是

resize() 扩容

结束

详细版:

第一步, 通过 hash 方法进一步扰动哈希值, 以减少哈希冲突。

```
static final int hash(Object key) {  
    int h;  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

No. 33 / 83

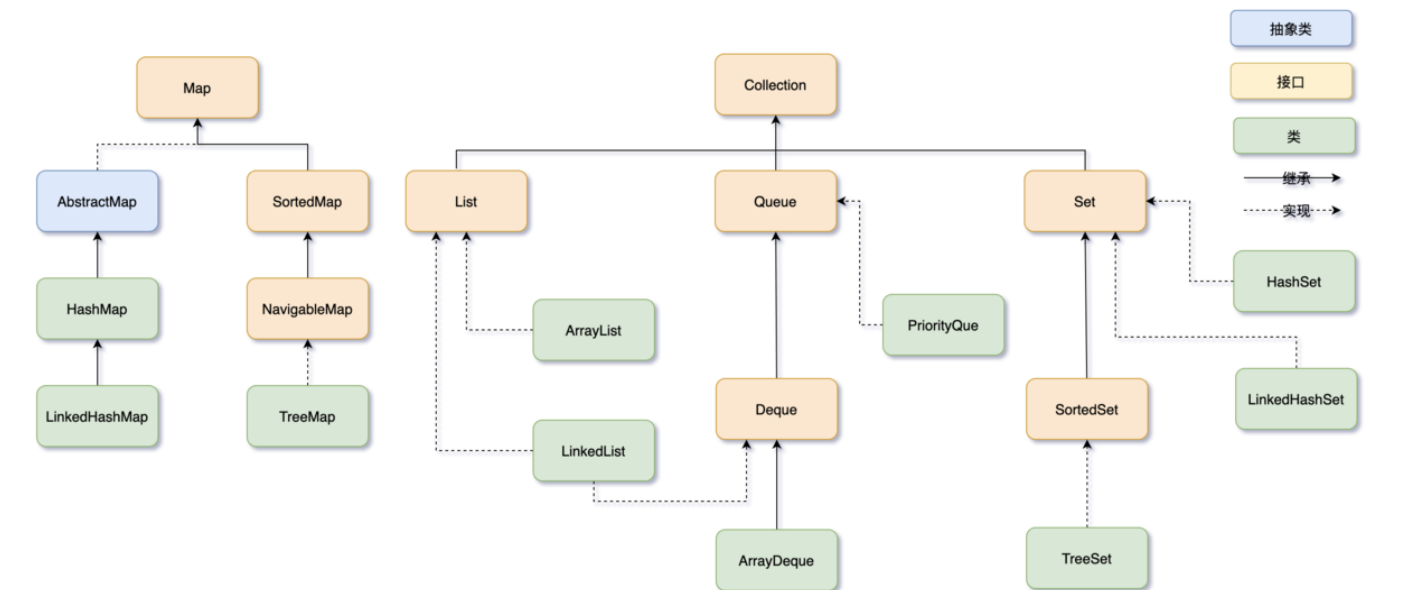
<< < 33 / 83 > >>

137.82%

引言

1. 🌟 说说有哪些常见的集合框架？

- 推荐阅读：[二哥的Java进阶之路：Java集合框架](#)
- 推荐阅读：[阻塞队列 BlockingQueue](#)。



集合框架可以分为两条大的支线：

①、第一条支线 Collection，主要由 List、Set、Queue 组成：

- List 代表有序、可重复的集合，典型代表就是封装了动态数组的 [ArrayList](#) 和封装了链表的 [LinkedList](#)；
- Set 代表无序、不可重复的集合，典型代表就是 HashSet 和 TreeSet；
- Queue 代表队列，典型代表就是双端队列 [ArrayDeque](#)，以及优先级队列 [PriorityQueue](#)。

②、第二条支线 Map，代表键值对的集合，典型代表就是 [HashMap](#)。

另外一个回答版本：

①、Collection 接口：最基本的集合框架表示方式，提供了添加、删除、清空等基本操作，它主要有三个子接口：

- `List`：一个有序的集合，可以包含重复的元素。实现类包括 ArrayList、LinkedList 等。
- `Set`：一个不包含重复元素的集合。实现类包括 HashSet、LinkedHashSet、TreeSet 等。
- `Queue`：一个用于保持元素队列的集合。实现类包括 PriorityQueue、ArrayDeque 等。

②、`Map` 接口：表示键值对的集合，一个键映射到一个值。键不能重复，每个键只能对应一个值。Map 接口的实现类包括 HashMap、LinkedHashMap、TreeMap 等。

集合框架有哪几个常用工具类？

集合框架位于 java.util 包下，提供了两个常用的工具类：

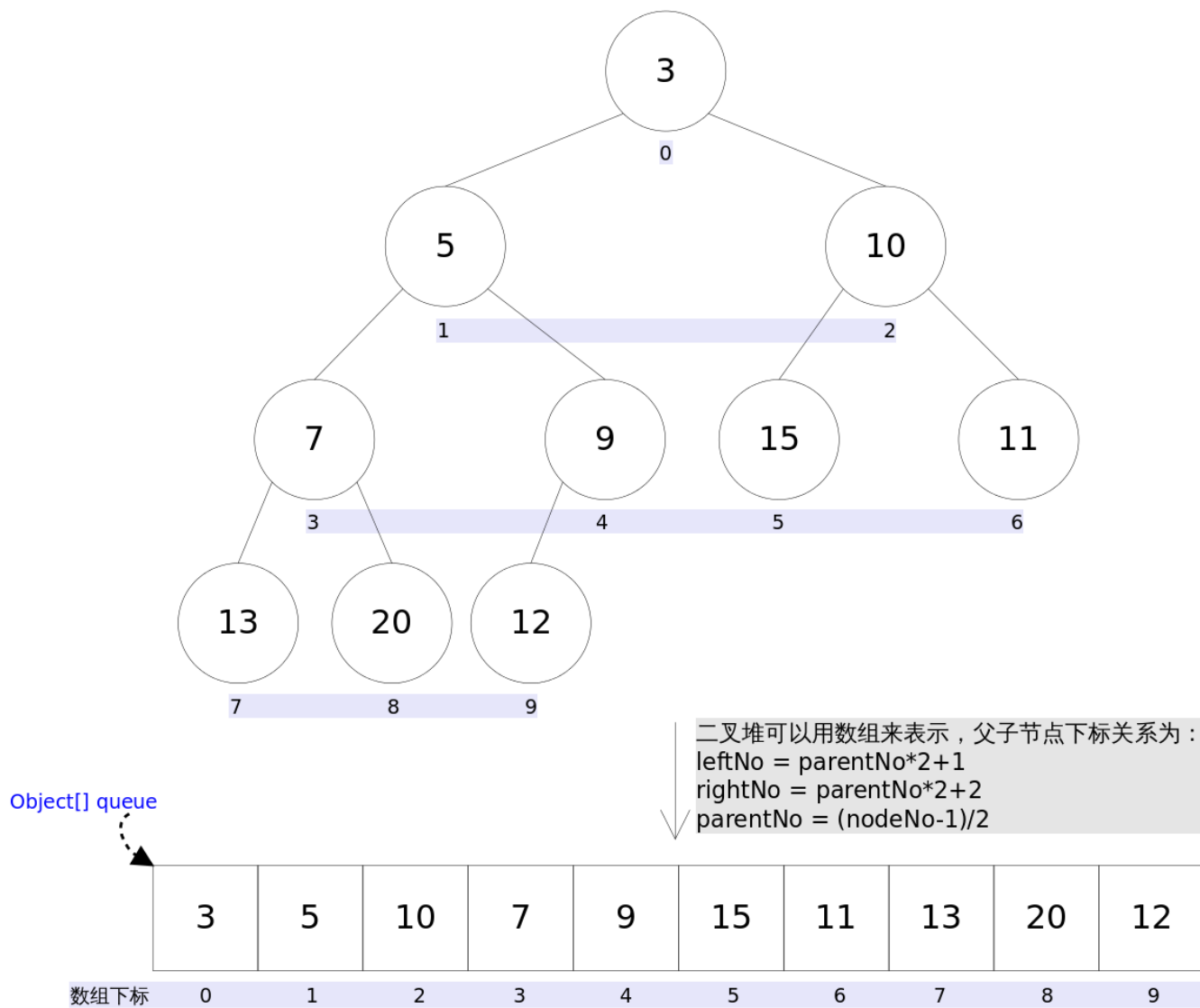
- [Collections](#)：提供了一些对集合进行排序、二分查找、同步的静态方法。
- [Arrays](#)：提供了一些对数组进行排序、打印、和 List 进行转换的静态方法。

简单介绍一下队列

Java 中的队列主要通过 Queue 接口和并发包下的 BlockingQueue 两个接口来实现。

优先级队列 PriorityQueue 实现了 Queue 接口，是一个无界队列，它的元素按照自然顺序排序或者 Comparator 比较器进行排序。

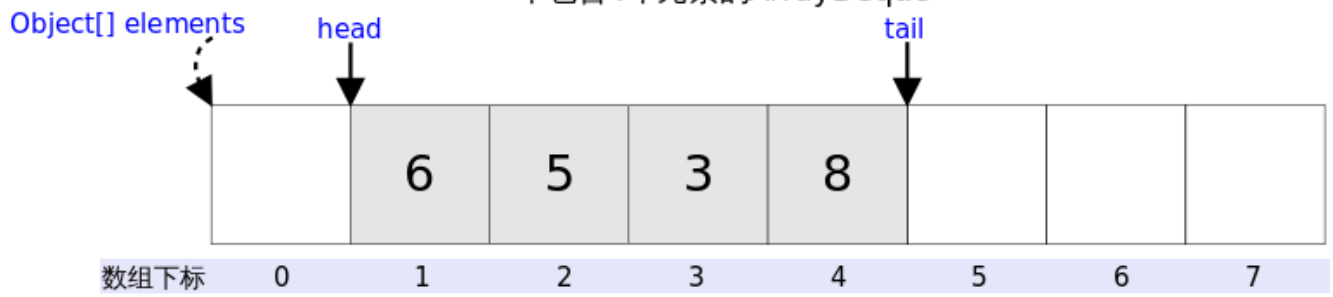
PriorityQueue通过用数组表示的小顶堆实现



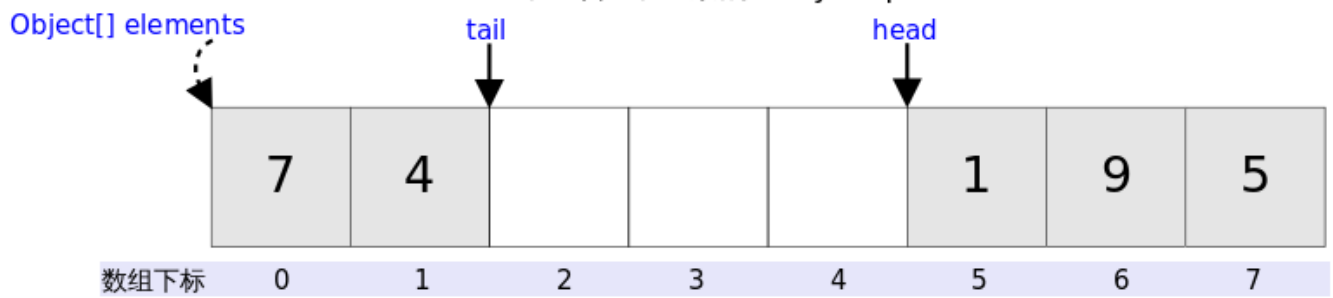
双端队列 ArrayDeque 也实现了 Queue 接口，是一个基于数组的，可以在两端插入和删除元素的队列。

ArrayDeque采用循环数组实现

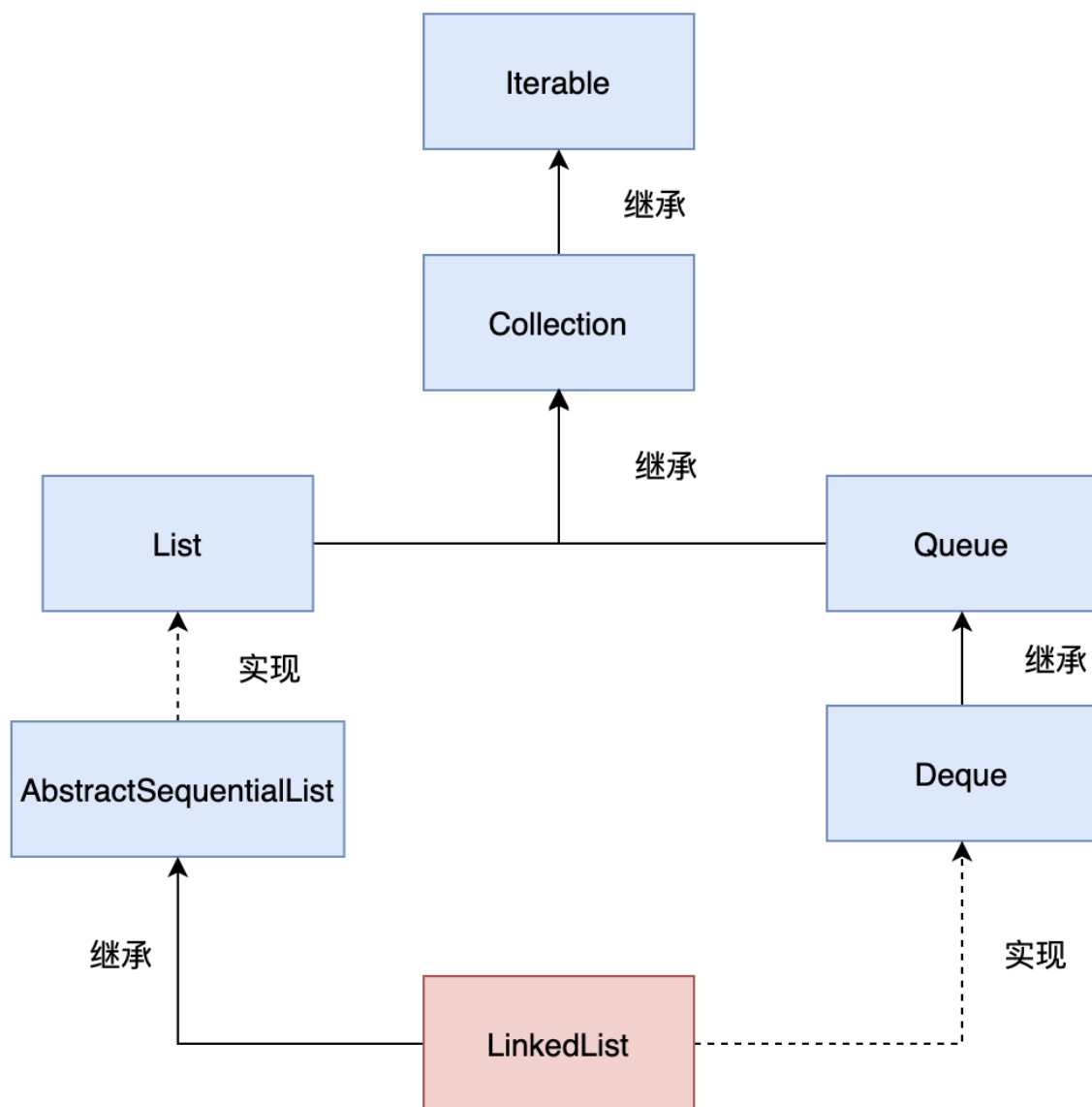
一个包含4个元素的ArrayDeque



一个包含5个元素的ArrayDeque



LinkedList 实现了 Queue 接口的子类 Deque，所以也可以当做双端队列来使用。



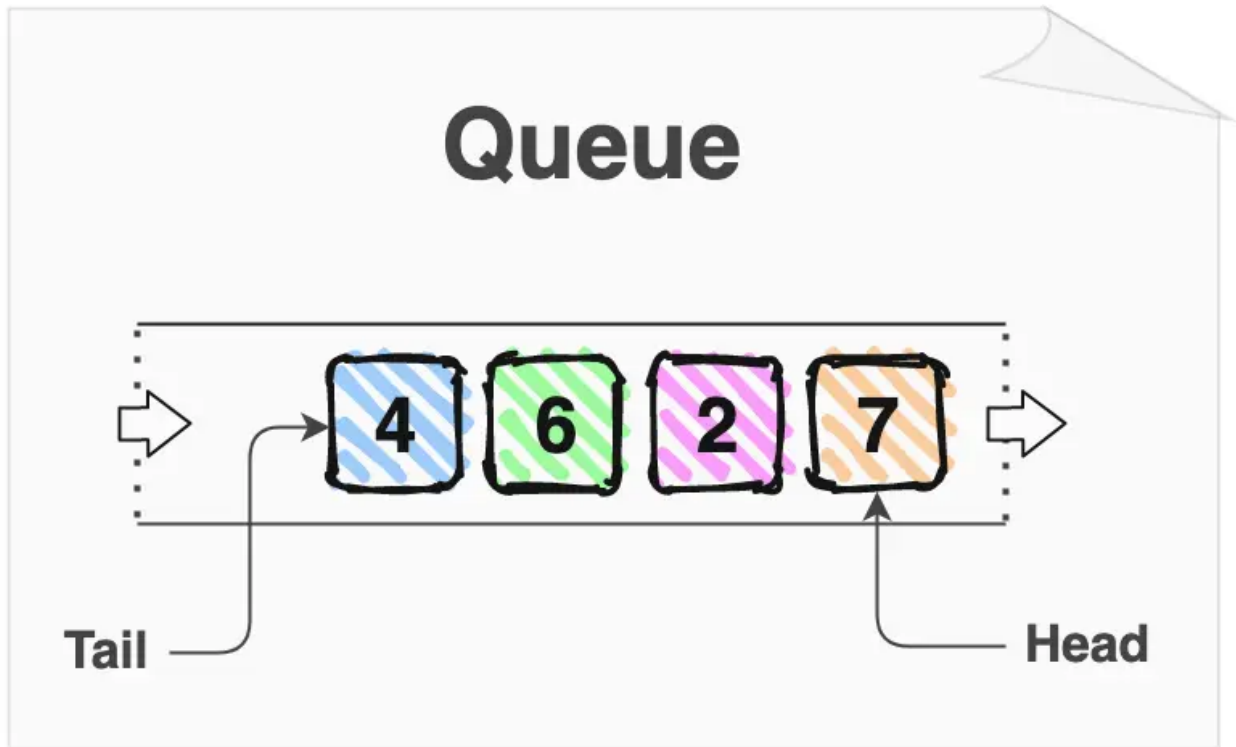
用过哪些集合类，它们的优劣？

我常用的集合类有 `ArrayList`、`LinkedList`、`HashMap`、`LinkedHashMap`。

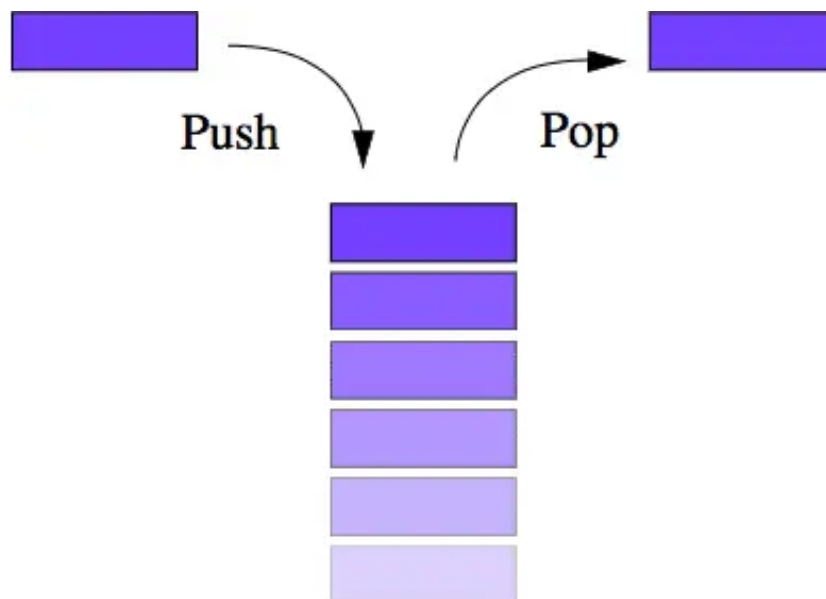
1. `ArrayList` 可以看作是一个动态数组，可以在需要时动态扩容数组的容量，只不过需要复制元素到新的数组。优点是访问速度快，可以通过索引直接查找到元素。缺点是插入和删除元素可能需要移动或者复制元素。
2. `LinkedList` 是一个双向链表，适合频繁的插入和删除操作。优点是插入和删除元素的时候只需要改变节点的前后指针，缺点是访问元素时需要遍历链表。
3. `HashMap` 是一个基于哈希表的键值对集合。优点是可以根据键的哈希值快速查找到值，但有可能会发生哈希冲突，并且不保留键值对的插入顺序。
4. `LinkedHashMap` 在 `HashMap` 的基础上增加了一个双向链表来保持键值对的插入顺序。

队列和栈的区别了解吗？

队列是一种先进先出（FIFO, First-In-First-Out）的数据结构，第一个加入队列的元素会成为第一个被移除的元素。



栈是一种后进先出（LIFO, Last-In-First-Out）的数据结构，最后一个加入栈的元素会成为第一个被移除的元素。

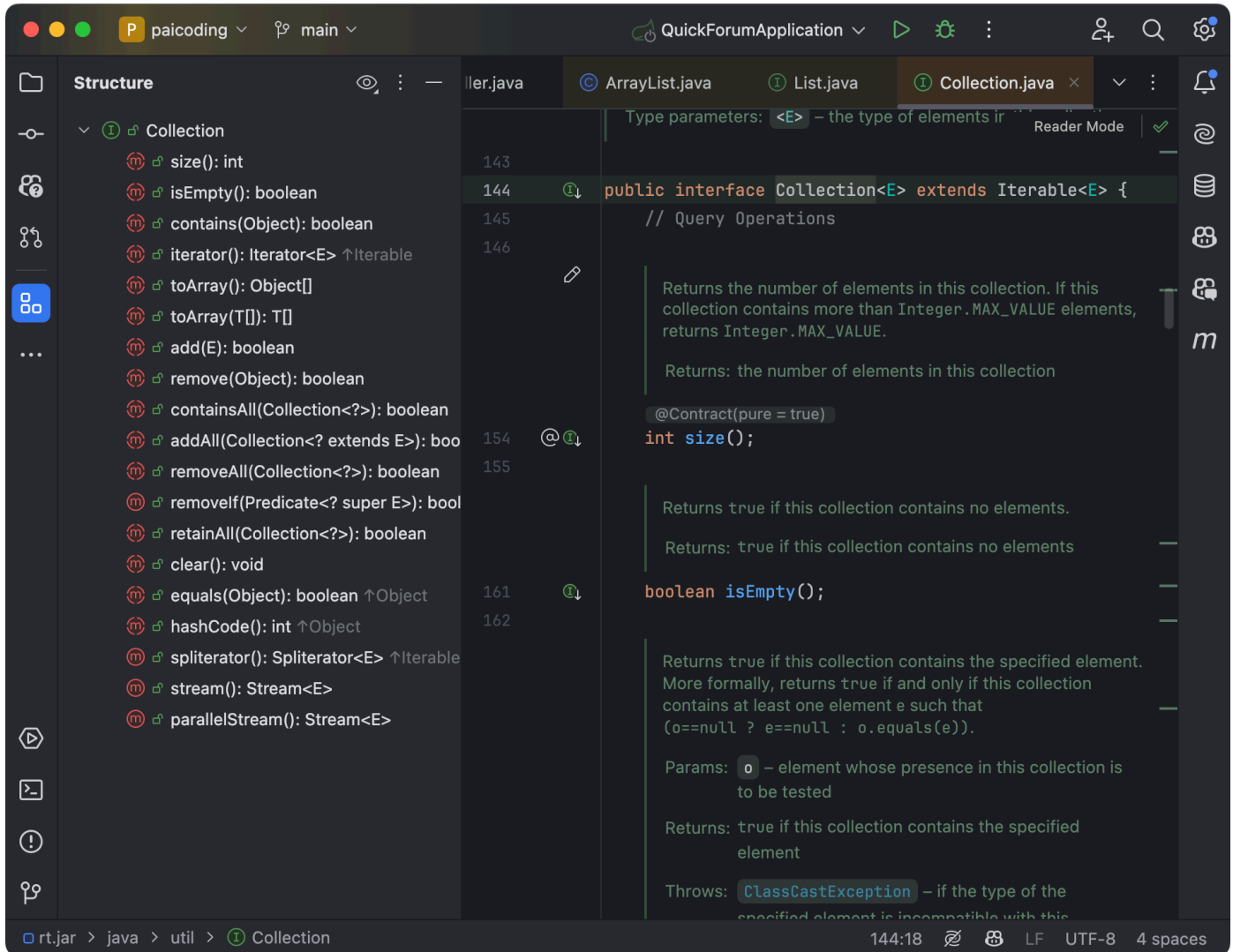


哪些是线程安全的容器？

像 Vector、Hashtable、ConcurrentHashMap、CopyOnWriteArrayList、ConcurrentLinkedQueue、ArrayBlockingQueue、LinkedBlockingQueue 都是线程安全的。

Collection 继承了哪些接口？

Collection 继承了 Iterable 接口，这意味着所有实现 Collection 接口的类都必须实现 `iterator()` 方法，之后就可以使用增强型 for 循环遍历集合中的元素了。



1. [Java 面试指南（付费）](#) 收录的用友金融一面原题：你了解哪些集合框架？
2. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Java 容器和 HashMap
3. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：你了解哪些集合？
4. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：知道哪些集合，讲讲 HashMap 和 TreeMap 的区别，讲讲两者应用场景的区别；讲一下有哪些队列，阻塞队列的阻塞是什么含义？
5. [Java 面试指南（付费）](#) 收录的农业银行面经同学 7 Java 后端面试原题：用过哪些集合类，它们的优劣
6. [Java 面试指南（付费）](#) 收录的华为 OD 面经同学 1 一面面试原题：队列和栈的区别了解吗？
7. [Java 面试指南（付费）](#) 收录的农业银行同学 1 面试原题：阻塞队列的实现方式
8. [Java 面试指南（付费）](#) 收录的小公司面经合集同学 1 Java 后端面试原题：Java 容器有哪些？List、Set 还有 Map 的区别？
9. [Java 面试指南（付费）](#) 收录的 360 面经同学 3 Java 后端技术一面面试原题：java 有哪些集合
10. [Java 面试指南（付费）](#) 收录的华为面经同学 11 面试原题：java 中的集合类型？哪些是线程安全的？
11. [Java 面试指南（付费）](#) 收录的招商银行面经同学 6 招银网络科技面试原题：Java 集合有哪些？
12. [Java 面试指南（付费）](#) 收录的用友面试原题：集合容器能列举几个吗？
13. [Java 面试指南（付费）](#) 收录的比亚迪面经同学 12 Java 技术面试原题：java 的集合介绍一下

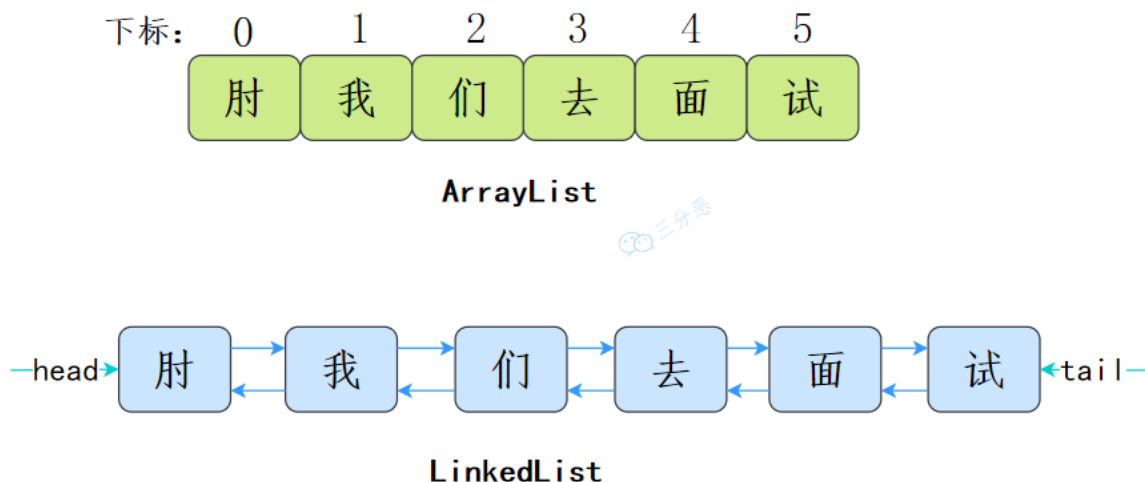
14. [Java 面试指南 \(付费\)](#) 收录的 OPPO 面经同学 1 面试原题：介绍Java的集合框架
15. [Java 面试指南 \(付费\)](#) 收录的vivo 面经同学 10 技术一面面试原题：Java中的集合有哪些

List

2. 🌟 ArrayList 和 LinkedList 有什么区别？

推荐阅读：[二哥的 Java 进阶之路：ArrayList 和 LinkedList](#)

ArrayList 是基于数组实现的，LinkedList 是基于链表实现的。



ArrayList 和 LinkedList 的用途有什么不同？

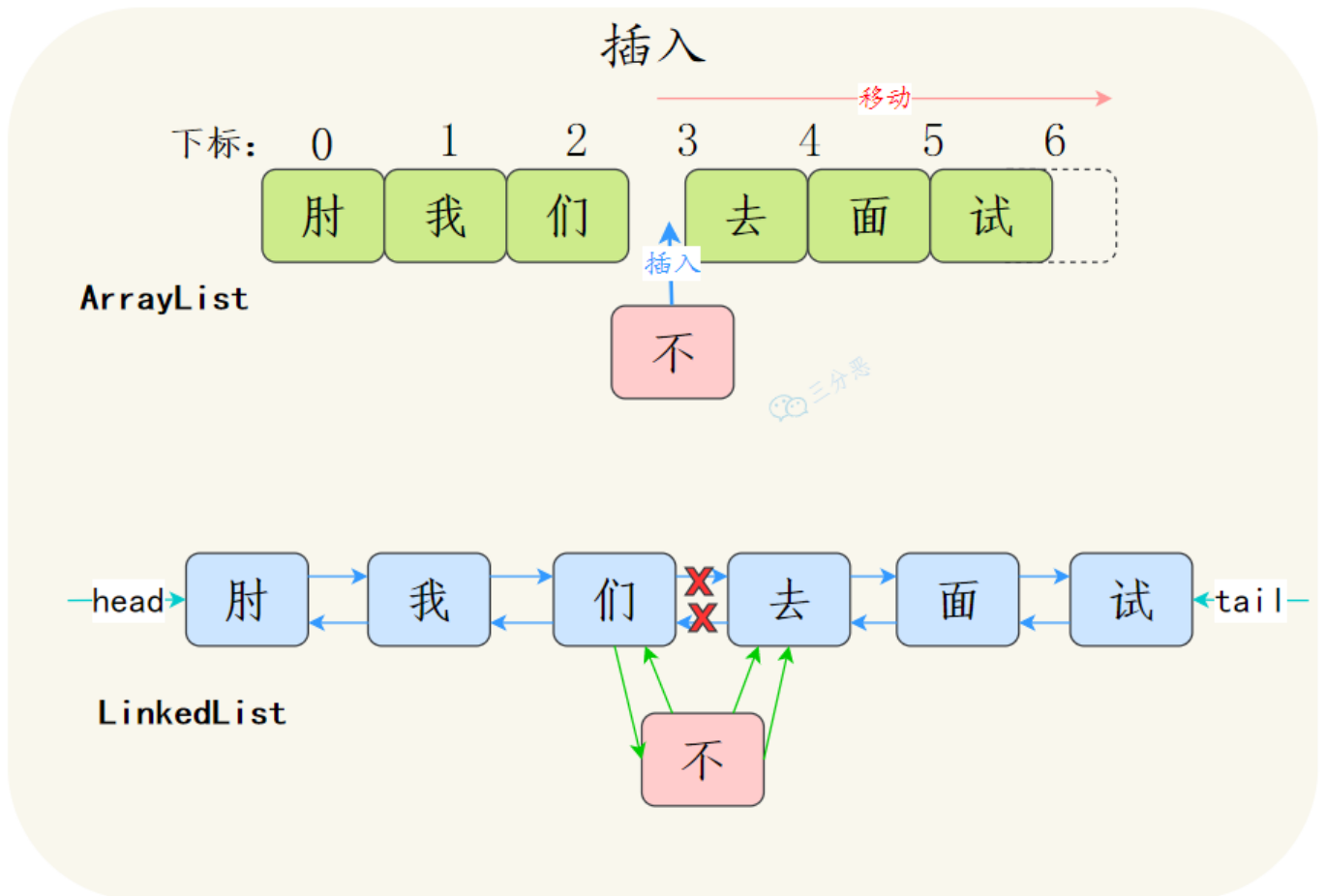
多数情况下，ArrayList 更利于查找，LinkedList 更利于增删。

①、由于 ArrayList 是基于数组实现的，所以 `get(int index)` 可以直接通过数组下标获取，时间复杂度是 $O(1)$ ；LinkedList 是基于链表实现的，`get(int index)` 需要遍历链表，时间复杂度是 $O(n)$ 。

当然，`get(E element)` 这种查找，两种集合都需要遍历通过 equals 比较获取元素，所以时间复杂度都是 $O(n)$ 。

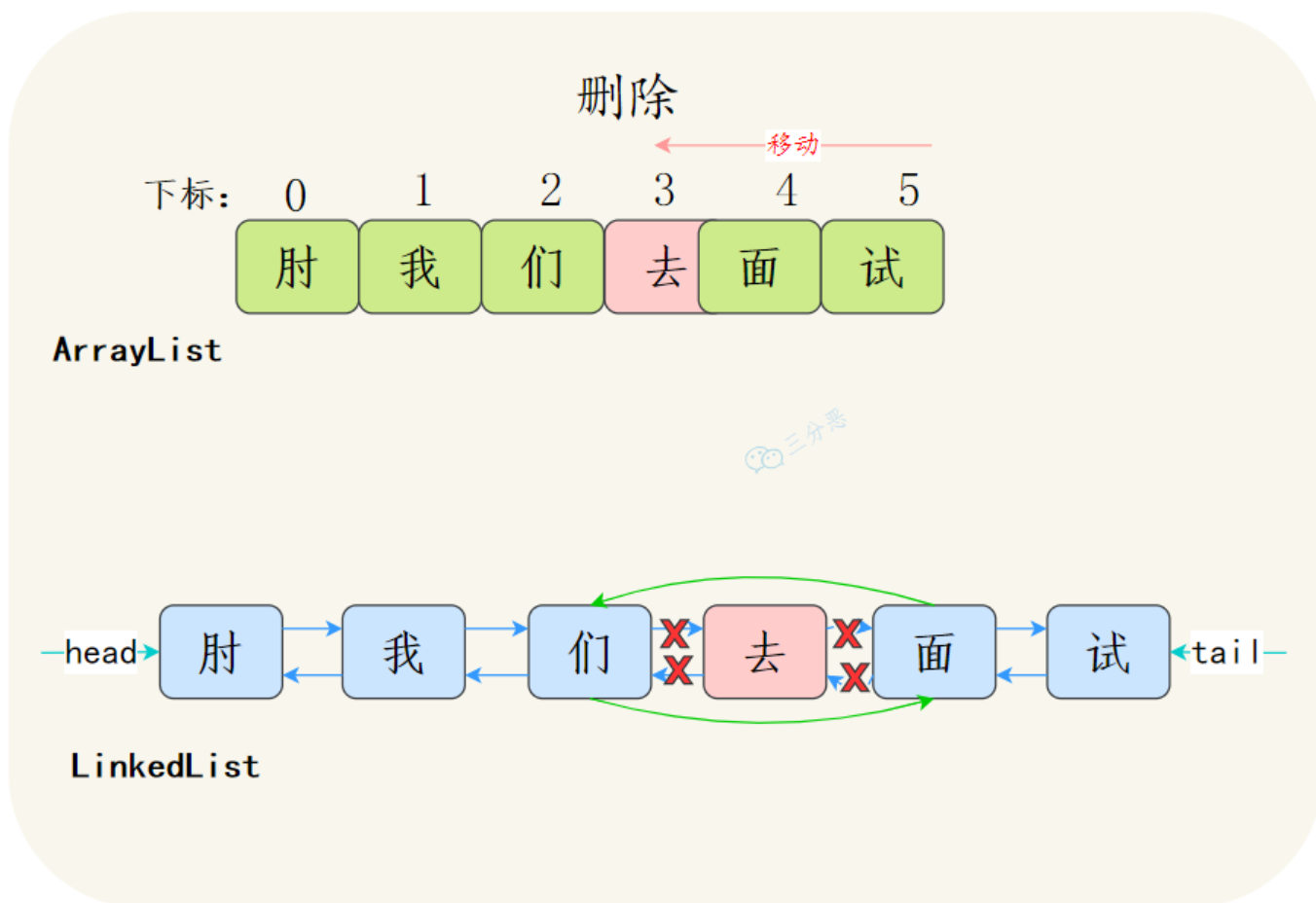
②、ArrayList 如果增删的是数组的尾部，时间复杂度是 $O(1)$ ；如果 add 的时候涉及到扩容，时间复杂度会上升到 $O(n)$ 。

但如果插入的是中间的位置，就需要把插入位置后的元素向前或者向后移动，甚至还有可能触发扩容，效率就会低很多，变成 $O(n)$ 。



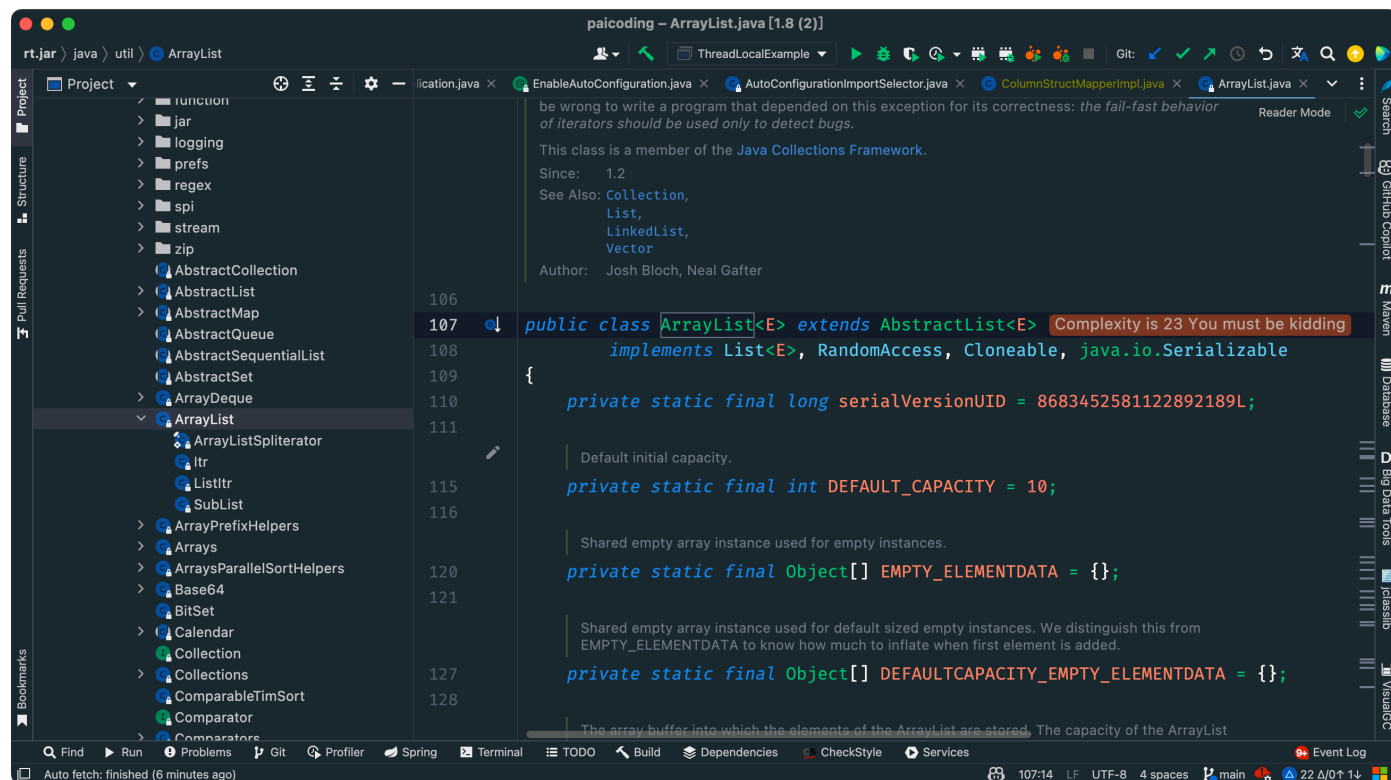
LinkedList 因为是链表结构，插入和删除只需要改变前置节点、后置节点和插入节点的引用，因此不需要移动元素。

如果是在链表的头部插入或者删除，时间复杂度是 $O(1)$ ；如果是在链表的中间插入或者删除，时间复杂度是 $O(n)$ ，因为需要遍历链表找到插入位置；如果是在链表的尾部插入或者删除，时间复杂度是 $O(1)$ 。

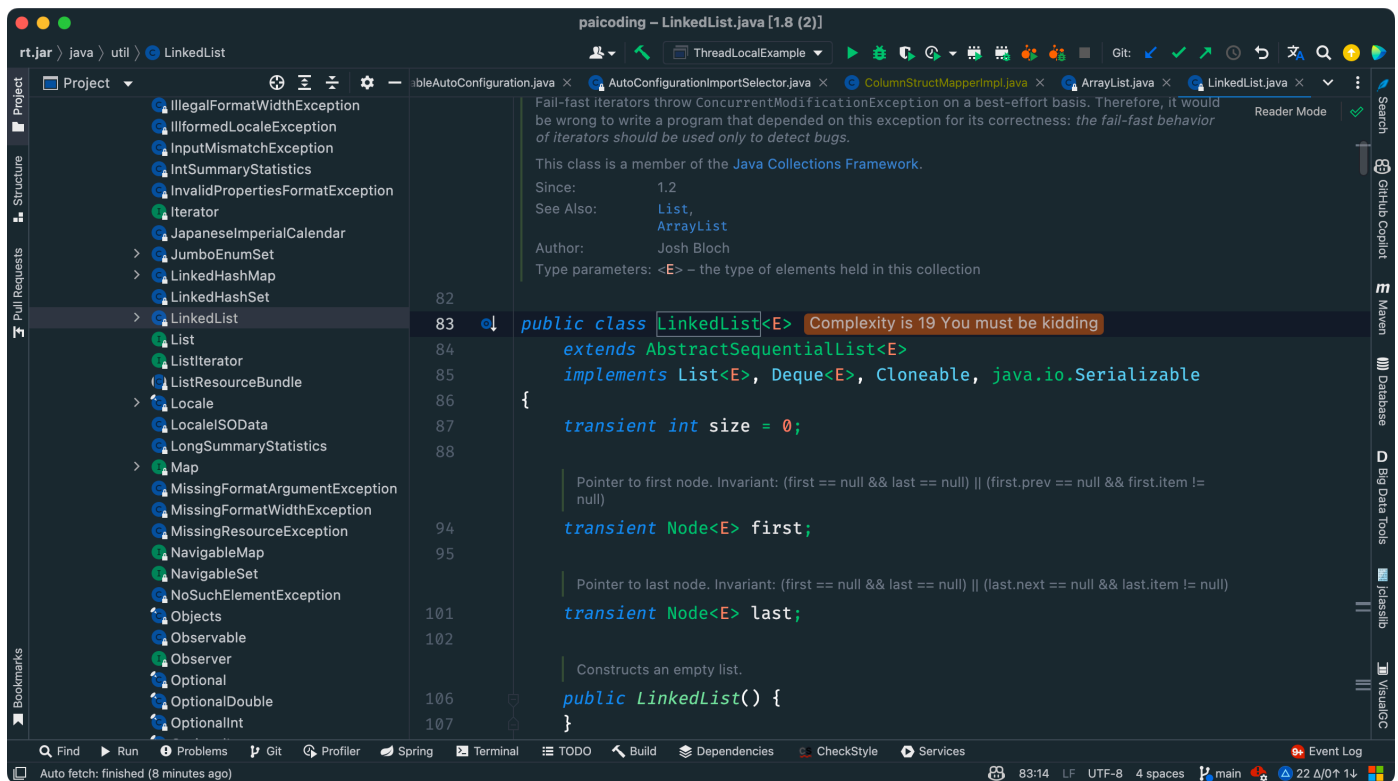


ArrayList 和 LinkedList 是否支持随机访问？

①、ArrayList 是基于数组的，也实现了 RandomAccess 接口，所以它支持随机访问，可以通过下标直接获取元素。

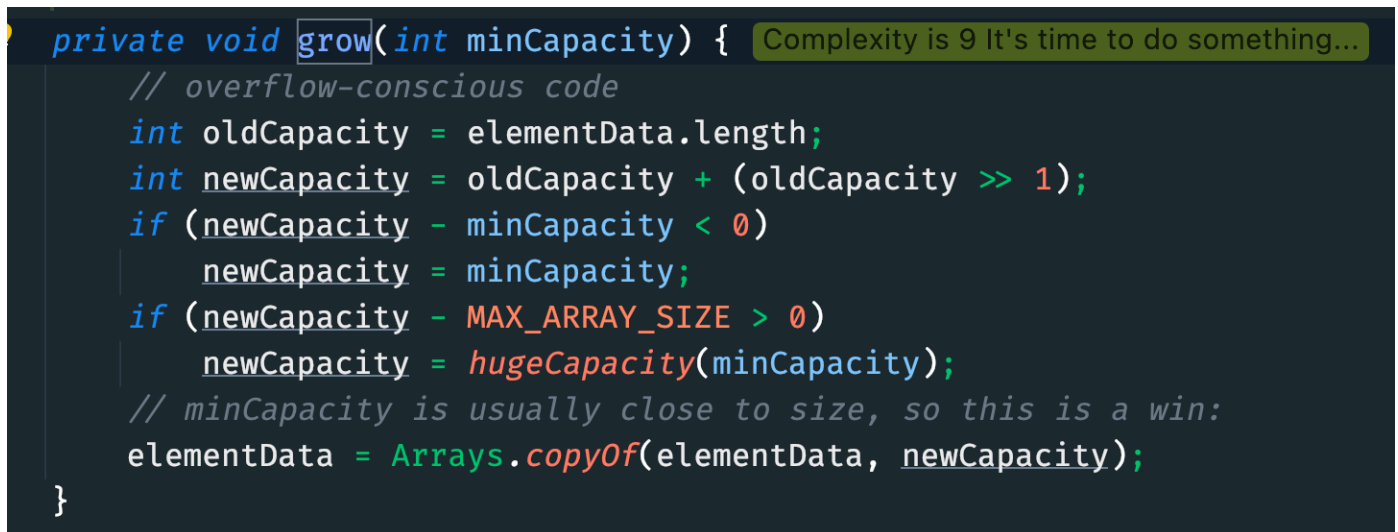


②、LinkedList 是基于链表的，所以它没法根据下标直接获取元素，不支持随机访问。



ArrayList 和 LinkedList 内存占用有何不同？

ArrayList 是基于数组的，是一块连续的内存空间，所以它的内存占用是比较紧凑的；但如果涉及到扩容，就会重新分配内存，空间是原来的 1.5 倍。



LinkedList 是基于链表的，每个节点都有一个指向下一个节点和上一个节点的引用，于是每个节点占用的内存空间比 ArrayList 稍微大一点。

ArrayList 和 LinkedList 的使用场景有什么不同？

ArrayList 适用于：

- 随机访问频繁：需要频繁通过索引访问元素的场景。
- 读取操作远多于写入操作：如存储不经常改变的列表。

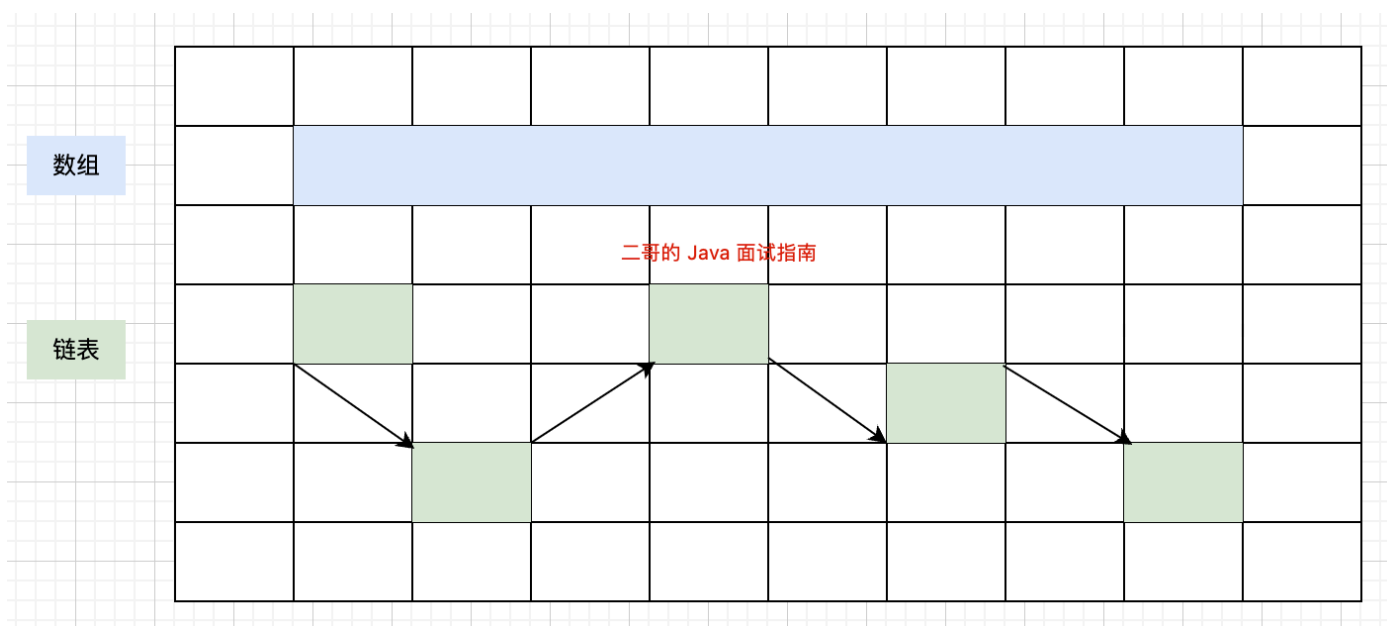
- 末尾添加元素：需要频繁在列表末尾添加元素的场景。

LinkedList 适用于：

- 频繁插入和删除：在列表中间频繁插入和删除元素的场景。
- 不需要快速随机访问：顺序访问多于随机访问的场景。
- 队列和栈：由于其双向链表的特性，LinkedList 可以实现队列（FIFO）和栈（LIFO）。

链表和数组有什么区别？

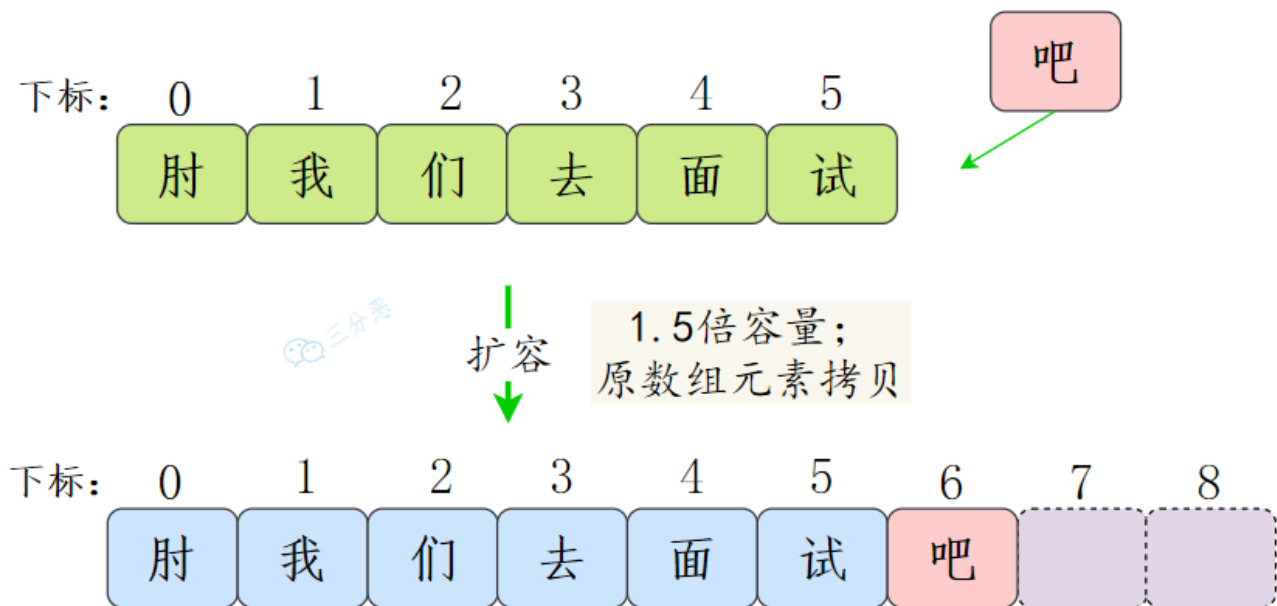
- 数组在内存中占用的是一块连续的存储空间，因此我们可以通过数组下标快速访问任意元素。数组在创建时必须指定大小，一旦分配内存，数组的大小就固定了。
- 链表的元素存储在于内存中的任意位置，每个节点通过指针指向下一个节点。



1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：ArrayList 和 LinkedList 的时间复杂度
2. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面试题原题：你了解哪些集合？
3. [Java 面试指南（付费）](#) 收录的小米面经同学 F 面试原题：ArrayList和LinkedList的区别和使用场景
4. [Java 面试指南（付费）](#) 收录的比亚迪面经同学 12 Java 技术面试原题：数组和链表的区别
5. [Java 面试指南（付费）](#) 收录的快手同学 2 一面试题原题：ArrayList和LinkedList区别
6. [Java 面试指南（付费）](#) 收录的得物面经同学 9 面试题题目原题：集合里面的arraylist和linkedlist的区别是什么？有何优缺点？

3.ArrayList 的扩容机制了解吗？

了解。当往 ArrayList 中添加元素时，会先检查是否需要扩容，如果当前容量+1 超过数组长度，就会进行扩容。



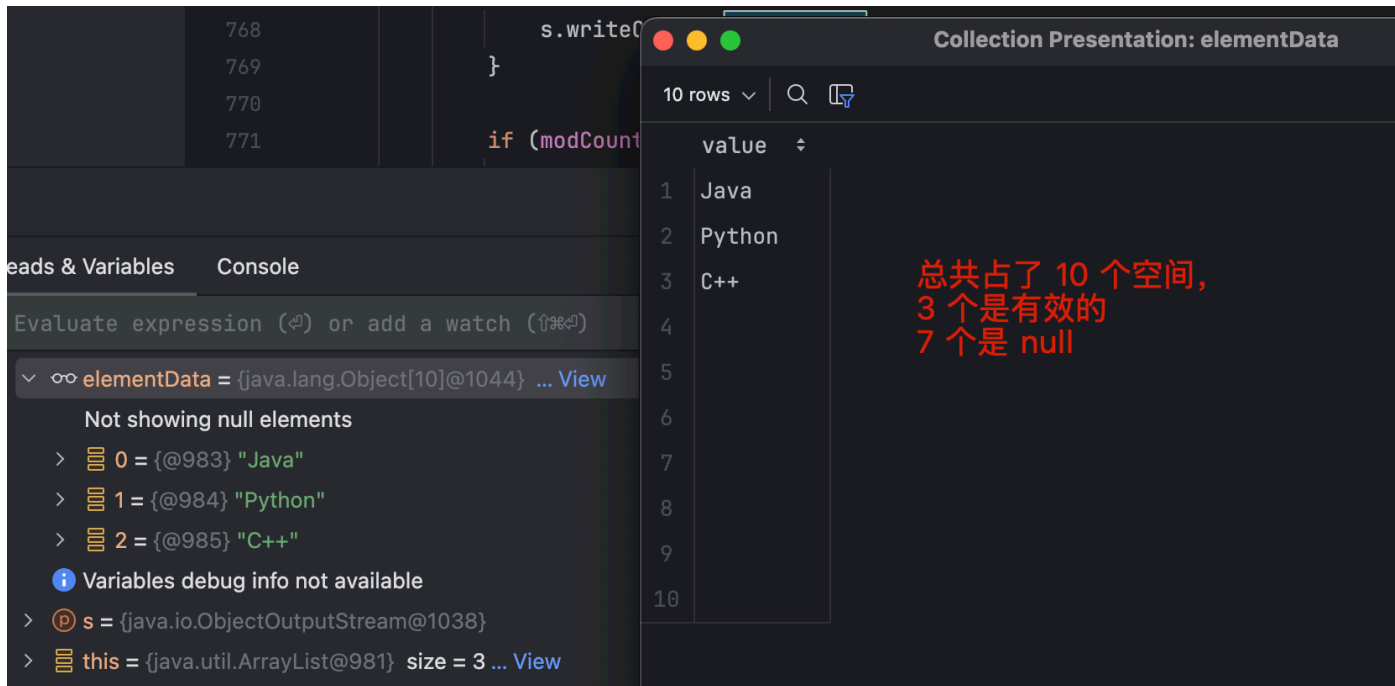
扩容后的新数组长度是原来的 1.5 倍，然后再把原数组的值拷贝到新数组中。

```
private void grow(int minCapacity) {
    // overflow-conscious code
    int oldCapacity = elementData.length;
    int newCapacity = oldCapacity + (oldCapacity >> 1);
    if (newCapacity - minCapacity < 0)
        newCapacity = minCapacity;
    if (newCapacity - MAX_ARRAY_SIZE > 0)
        newCapacity = hugeCapacity(minCapacity);
    // minCapacity is usually close to size, so this is a win:
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

1. [Java 面试指南 \(付费\)](#) 收录的联想面经同学 7 面试原题：Java 集合类介绍，挑一个讲原理。

4.ArrayList 怎么序列化的知道吗？

在 ArrayList 中，writeObject 方法被重写了，用于自定义序列化逻辑：只序列化有效数据，因为 elementData 数组的容量一般大于实际的元素数量，声明的时候也加了 transient 关键字。



为什么 ArrayList 不直接序列化元素数组呢？

出于效率的考虑，数组可能长度 100，但实际只用了 50，剩下的 50 没用到，也就不需要序列化。

```
private void writeObject(java.io.ObjectOutputStream s)
    throws java.io.IOException {
    // 将当前 ArrayList 的结构进行序列化
    int expectedModCount = modCount;
    s.defaultWriteObject(); // 序列化非 transient 字段
    // 序列化数组的大小
    s.writeInt(size);
    // 序列化每个元素
    for (int i = 0; i < size; i++) {
        s.writeObject(elementData[i]);
    }
    // 检查是否在序列化期间发生了并发修改
    if (modCount != expectedModCount) {
        throw new ConcurrentModificationException();
    }
}
```

5.快速失败fail-fast了解吗？

fail—fast 是 Java 集合的一种错误检测机制。

在用迭代器遍历集合对象时，如果线程 A 遍历过程中，线程 B 对集合对象的内容进行了修改，就会抛出 Concurrent Modification Exception。

迭代器在遍历时直接访问集合中的内容，并且在遍历过程中使用一个 `modCount` 变量。集合在被遍历期间如果内容发生变化，就会改变 `modCount` 的值。每当迭代器使用 `hashNext()/next()` 遍历下一个元素之前，都会检测 `modCount` 变量是否为 `expectedmodCount` 值，是的话就返回遍历；否则抛出异常，终止遍历。

异常的抛出条件是检测到 `modCount != expectedmodCount` 这个条件。如果集合发生变化时修改 `modCount` 值刚好又设置为了 `expectedmodCount` 值，则异常不会抛出。因此，不能依赖于这个异常是否抛出而进行并发操作的编程，这个异常只建议用于检测并发修改的 bug。

`java.util` 包下的集合类都是快速失败的，不能在多线程下发生并发修改（迭代过程中被修改），比如 `ArrayList` 类。

什么是安全失败（fail—safe）呢？

采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，在拷贝的集合上进行遍历。

原理：由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到，所以不会触发 `Concurrent Modification Exception`。

缺点：基于拷贝内容的优点是避免了 `Concurrent Modification Exception`，但同样地，迭代器并不能访问到修改后的内容，即：迭代器遍历的是开始遍历那一刻拿到的集合拷贝，在遍历期间原集合发生的修改迭代器是不知道的。

场景：`java.util.concurrent` 包下的容器都是安全失败，可以在多线程下并发使用，并发修改，比如 `CopyOnWriteArrayList` 类。

6.有哪几种实现 ArrayList 线程安全的方法？

常用的有两种。

可以使用 `Collections.synchronizedList()` 方法，它可以返回一个线程安全的 `List`。

```
SynchronizedList list = Collections.synchronizedList(new ArrayList());
```

内部是通过 [synchronized 关键字](#)加锁来实现的。

也可以直接使用 [CopyOnWriteArrayList](#)，它是线程安全的 `ArrayList`，遵循写时复制的原则，每当对列表进行修改时，都会创建一个新副本，这个新副本会替换旧的列表，而对旧列表的所有读取操作仍然在原有的列表上进行。

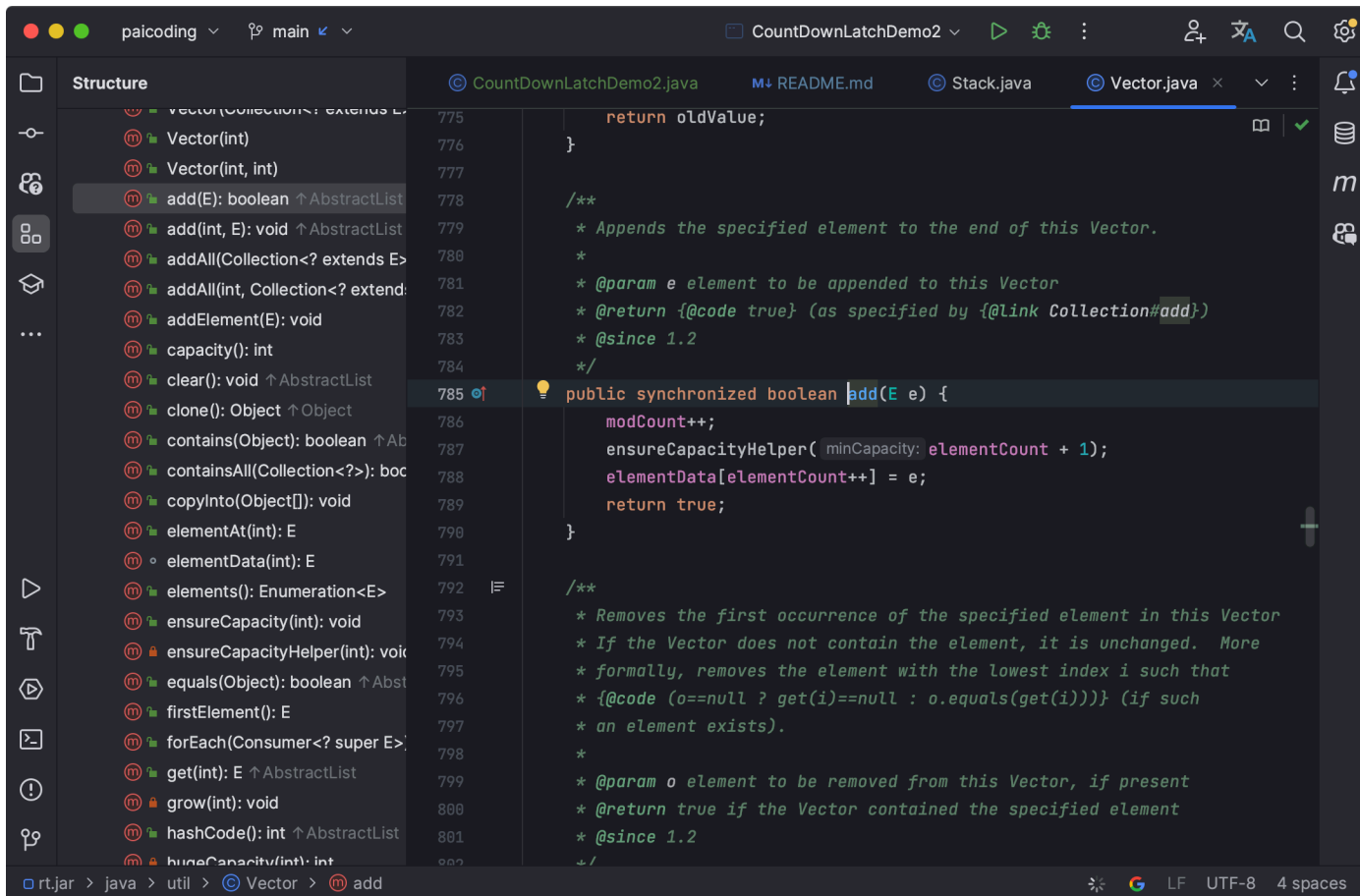
```
CopyOnWriteArrayList list = new CopyOnWriteArrayList();
```

通俗的讲，`CopyOnWrite` 就是当我们往一个容器添加元素的时候，不直接往容器中添加，而是先复制出一个新的容器，然后在新的容器里添加元素，添加完之后，再将原容器的引用指向新的容器。多个线程在读的时候，不需要加锁，因为当前容器不会添加任何元素。这样就实现了线程安全。

ArrayList 和 Vector 的区别？

`Vector` 属于 JDK 1.0 时期的遗留类，不推荐使用，仍然保留着是因为 Java 希望向后兼容。

`ArrayList` 是在 JDK 1.2 时引入的，用于替代 `Vector` 作为主要的非同步动态数组实现。因为 `Vector` 所有的方法都使用了 `synchronized` 关键字进行同步，所以单线程环境下效率较低。



1. [Java 面试指南（付费）](#) 收录的招商银行面经同学 6 招银网络科技面试原题：线程不安全的集合变成线程安全的方法？
2. [Java 面试指南（付费）](#) 收录的 比亚迪面经同学2面试原题：ArrayList 和 vector 的区别

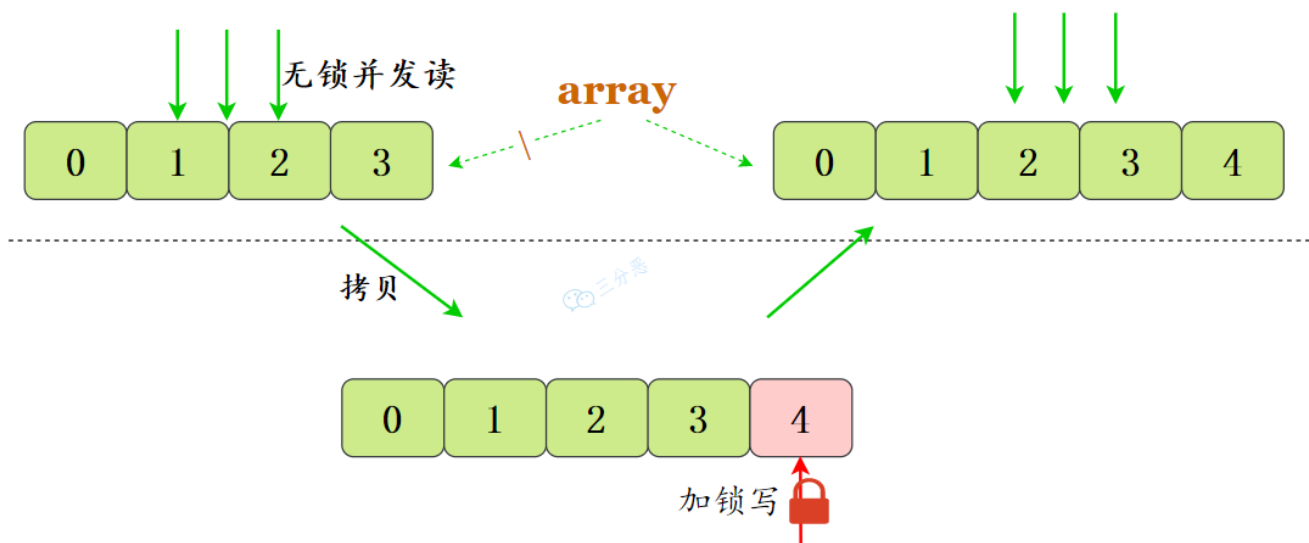
7.CopyOnWriteArrayList 了解多少？

CopyOnWriteArrayList 就是线程安全版本的 ArrayList。

`CopyOnWrite` ——写时复制，已经明示了它的原理。

CopyOnWriteArrayList 采用了一种读写分离的并发策略。CopyOnWriteArrayList 容器允许并发读，读操作是无锁的。至于写操作，比如说向容器中添加一个元素，首先将当前容器复制一份，然后在新副本上执行写操作，结束之后再将原容器的引用指向新容器。

- ①将原数组拷贝一份
- ②写操作在副本上，加锁；此时读操作在原数组上
- ③写完将元素数组指向副本



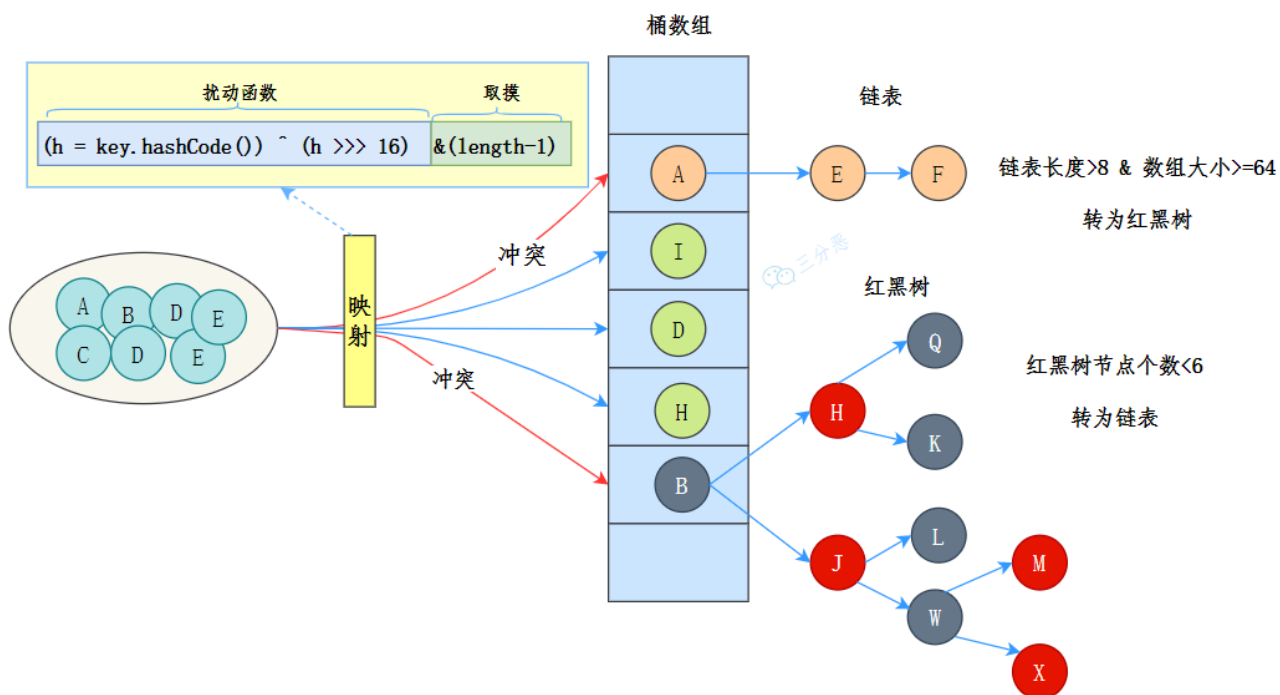
Map

Map 中最重要的就是 HashMap 了，面试基本被问出包浆了，一定要好好准备。

8. 🌟能说一下 HashMap 的底层数据结构吗？

推荐阅读：[二哥的 Java 进阶之路：详解 HashMap](#)

JDK 8 中 HashMap 的数据结构是 数组 + 链表 + 红黑树。



数组用来存储键值对，每个键值对可以通过索引直接拿到，索引是通过键的哈希值进行进一步的 `hash()` 处理得到的。

当多个键经过哈希处理后得到相同的索引时，需要通过链表来解决哈希冲突——将具有相同索引的键值对通过链表存储起来。

不过，链表过长时，查询效率会比较低，于是当链表的长度超过 8 时（且数组的长度大于 64），链表就会转换为红黑树。红黑树的查询效率是 $O(\log n)$ ，比链表的 $O(n)$ 要快。

`hash()` 方法的目标是尽量减少哈希冲突，保证元素能够均匀地分布在数组的每个位置上。

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

如果键的哈希值已经在数组中存在，其对应的值将被新值覆盖。

HashMap 的初始容量是 16，随着元素的不断添加，HashMap 就需要进行扩容，阈值是 `capacity * loadFactor`，`capacity` 为容量，`loadFactor` 为负载因子，默认为 0.75。

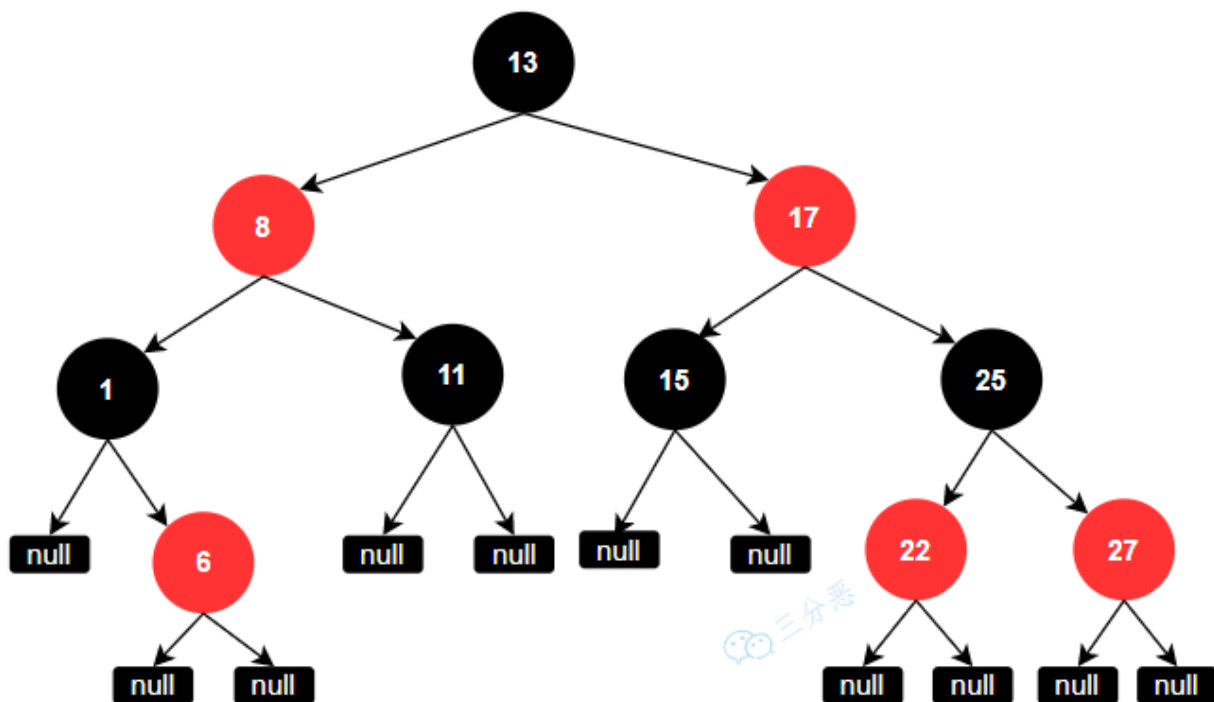
扩容后的数组大小是原来的 2 倍，然后把原来的元素重新计算哈希值，放到新的数组中。

1. [Java 面试指南（付费）](#) 收录的小米 25 届日常实习一面原题：讲一讲 HashMap 的原理
2. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Java 容器和 HashMap
3. [Java 面试指南（付费）](#) 收录的华为一面原题：说下 Redis 和 HashMap 的区别
4. [Java 面试指南（付费）](#) 收录的国企面试原题：说说 HashMap 的底层数据结构，链表和红黑树的转换，HashMap 的长度
5. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：说一下 HashMap 数据库结构和一些重要参数
6. [Java 面试指南（付费）](#) 收录的腾讯云智面经同学 16 一面面试原题：HashMap 的底层实现，它为什么是线程不安全的？
7. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：HashMap 的结构？
8. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？
9. [Java 面试指南（付费）](#) 收录的腾讯面经同学 27 云后台技术一面面试原题：Hashmap的底层？为什么链表要变成红黑树？为什么不用平衡二叉树？

9.你对红黑树了解多少？

红黑树是一种自平衡的二叉查找树：

1. 每个节点要么是红色，要么是黑色；
2. 根节点永远是黑色；
3. 所有的叶子节点都是是黑色的（下图中的 NULL 节点）；
4. 红色节点的子节点一定是黑色的；
5. 从任一节点到其每个叶子的所有简单路径都包含相同数目的黑色节点。



为什么不用二叉树？

二叉树是最基本的树结构，每个节点最多有两个子节点，但是二叉树容易出现极端情况，比如插入的数据是有序的，那么二叉树就会退化成链表，查询效率就会变成 $O(n)$ 。

为什么不用平衡二叉树？

平衡二叉树比红黑树的要求更高，每个节点的左右子树的高度最多相差 1，这种高度的平衡保证了极佳的查找效率，但在进行插入和删除操作时，可能需要频繁地进行旋转来维持树的平衡，维护成本更高。

为什么用红黑树？

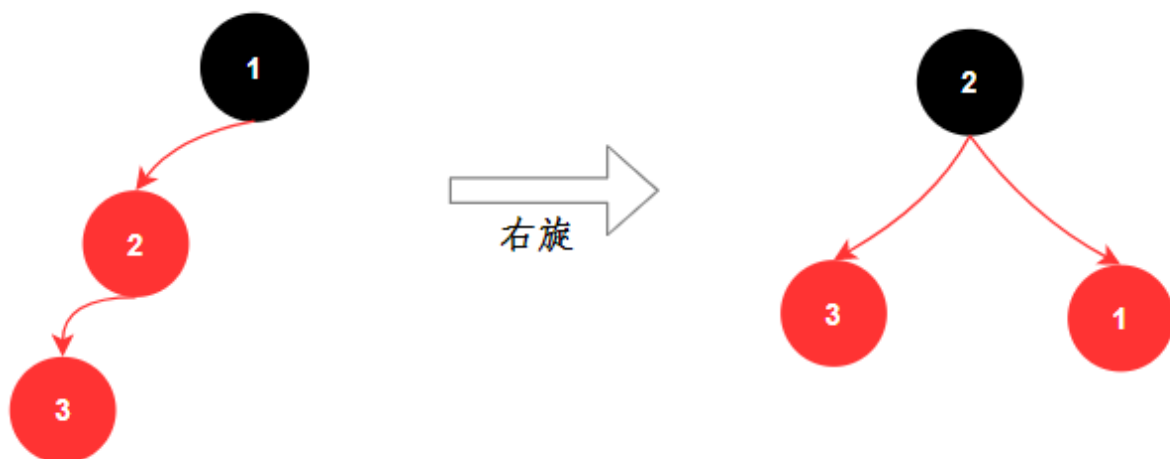
链表的查找时间复杂度是 $O(n)$ ，当链表长度较长时，查找性能会下降。红黑树是一种折中的方案，查找、插入、删除的时间复杂度都是 $O(\log n)$ 。

1. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：HashMap 为什么用红黑树，链表转数条件，红黑树插入删除规则
2. [Java 面试指南（付费）](#) 收录的快手同学 2 一面面试原题：为什么HashMap采用红黑树？

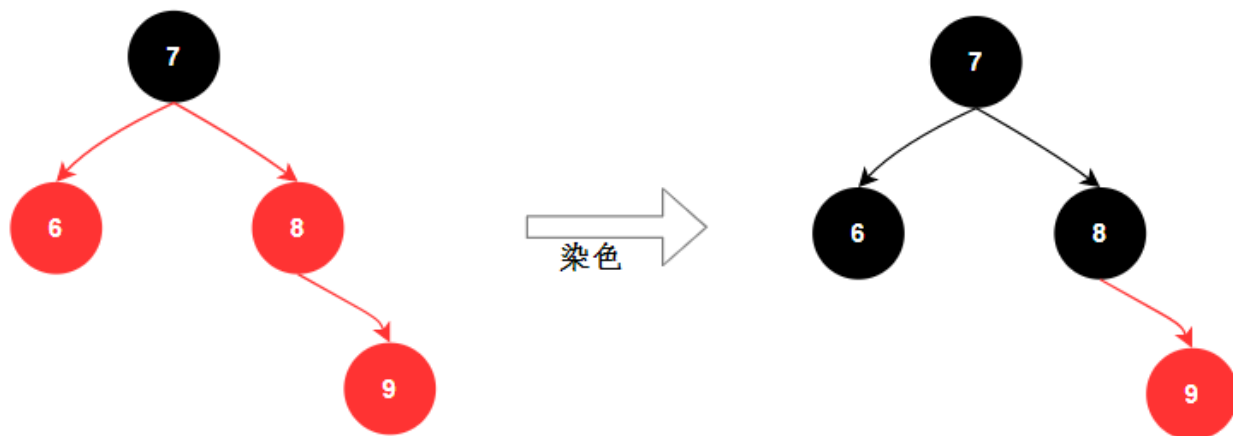
10.红黑树怎么保持平衡的？

旋转 和 染色。

- ①、通过左旋和右旋来调整树的结构，避免某一侧过深。



②、染色，修复红黑规则，从而保证树的高度不会失衡。



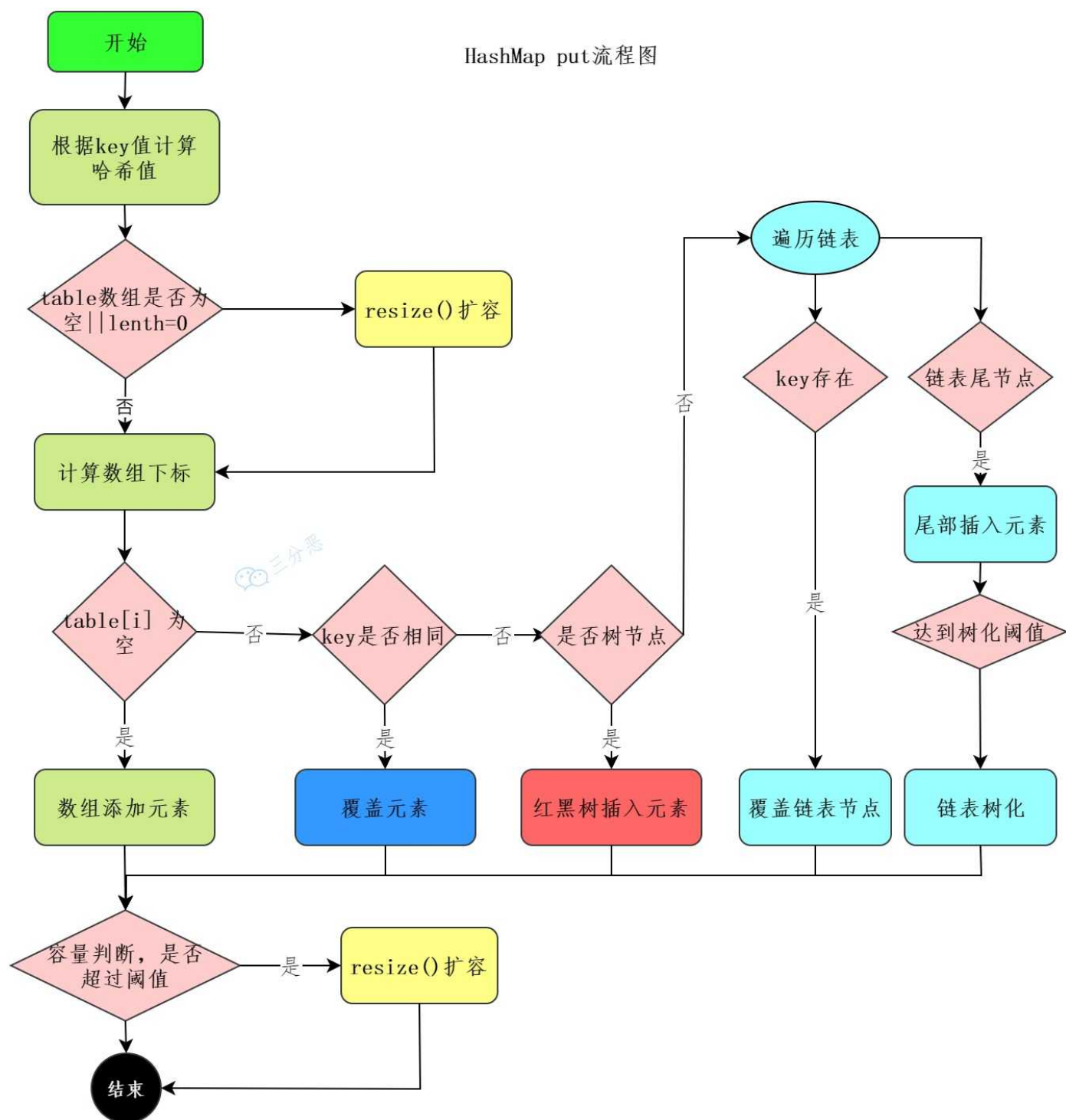
1. [Java 面试指南 \(付费\)](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：HashMap 为什么用红黑树，链表转数条件，红黑树插入删除规则

memo：2025 年 1 月 6 日修改到此。

11. 🌟 HashMap 的 put 流程知道吗？

哈希寻址 → 处理哈希冲突（链表还是红黑树）→ 判断是否需要扩容 → 插入/覆盖节点。

HashMap put流程图



详细版：

第一步，通过 hash 方法进一步扰动哈希值，以减少哈希冲突。

```

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
  
```


第二步，进行第一次的数组扩容；并使用哈希值和数组长度进行取模运算，确定索引位置。

```
if ((tab = table) == null || (n = tab.length) == 0)
    n = (tab = resize()).length;

if ((p = tab[i = (n - 1) & hash]) == null)
    tab[i] = newNode(hash, key, value, null);
```

如果当前位置为空，直接将键值对插入该位置；否则判断当前位置的第一个节点是否与新节点的 key 相同，如果相同直接覆盖 value，如果不同，说明发生哈希冲突。

如果是链表，将新节点添加到链表的尾部；如果链表长度大于等于 8，则将链表转换为红黑树。

```
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}

final V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
    Node<K,V>[] tab; Node<K,V> p; int n, i;
    // 如果 table 为空，先进行初始化
    if ((tab = table) == null || (n = tab.length) == 0)
        n = (tab = resize()).length;

    // 计算索引位置，并找到对应的桶
    if ((p = tab[i = (n - 1) & hash]) == null)
        tab[i] = newNode(hash, key, value, null); // 如果桶为空，直接插入
    else {
        Node<K,V> e; K k;
        // 检查第一个节点是否匹配
        if (p.hash == hash && ((k = p.key) == key || (key != null && key.equals(k))))
            e = p; // 覆盖
        // 如果是树节点，放入树中
        else if (p instanceof TreeNode)
            e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);
        // 如果是链表，遍历插入到尾部
        else {
            for (int binCount = 0; ; ++binCount) {
                if ((e = p.next) == null) {
                    p.next = newNode(hash, key, value, null);
                    // 如果链表长度达到阈值，转换为红黑树
                    if (binCount >= TREEIFY_THRESHOLD - 1)
                        treeifyBin(tab, hash);
                    break;
                }
                if (e.hash == hash && ((k = e.key) == key || (key != null &&
key.equals(k))))
                    break; // 覆盖
                p = e;
            }
        }
        if (e != null) { // 如果找到匹配的 key，则覆盖旧值
```

```

        V oldValue = e.value;
        if (!onlyIfAbsent || oldValue == null)
            e.value = value;
        afterNodeAccess(e);
        return oldValue;
    }
}
++modCount; // 修改计数器
if (++size > threshold)
    resize(); // 检查是否需要扩容
afterNodeInsertion(evict);
return null;
}

```

每次插入新元素后，检查是否需要扩容，如果当前元素个数大于阈值（`capacity * loadFactor`），则进行扩容，扩容后的数组大小是原来的 2 倍；并且重新计算每个节点的索引，进行数据重新分布。

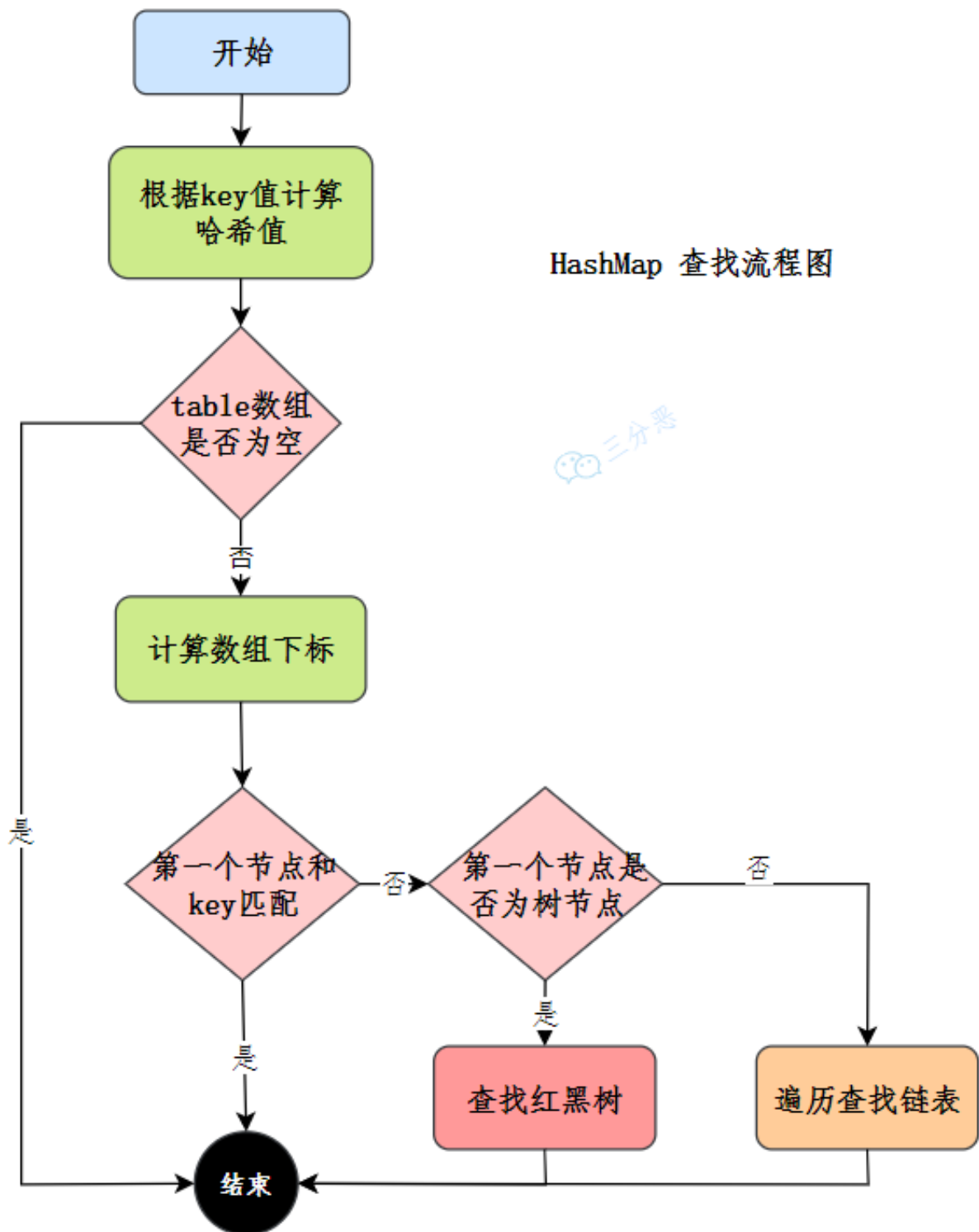
只重写元素的 equals 方法没重写 hashCode，put 的时候会发生什么？

如果只重写 equals 方法，没有重写 hashCode 方法，那么会导致 equals 相等的两个对象，hashCode 不相等，这样的话，两个对象会被 put 到数组中不同的位置，导致 get 的时候，无法获取到正确的值。

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：hashCode 和 equals 方法只重写一个行不行，只重写 equals 没重写 hashCode，map put 的时候会发生什么
2. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：HashMap 的 put 过程
3. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？
4. [Java 面试指南（付费）](#) 收录的快手同学 2 一面面试原题：HashMap 存放元素流程

12.HashMap 怎么查找元素的呢？

通过哈希值定位索引 → 定位桶 → 检查第一个节点 → 遍历链表或红黑树查找 → 返回结果。



13.HashMap 的 hash 函数是怎么设计的？

先拿到 key 的哈希值，是一个 32 位的 int 类型数值，然后再让哈希值的高 16 位和低 16 位进行异或操作，这样能保证哈希分布均匀。

```
static final int hash(Object key) {
    int h;
    // 如果 key 为 null, 返回 0; 否则, 使用 hashCode 并进行扰动
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

14.为什么 hash 函数能减少哈希冲突?

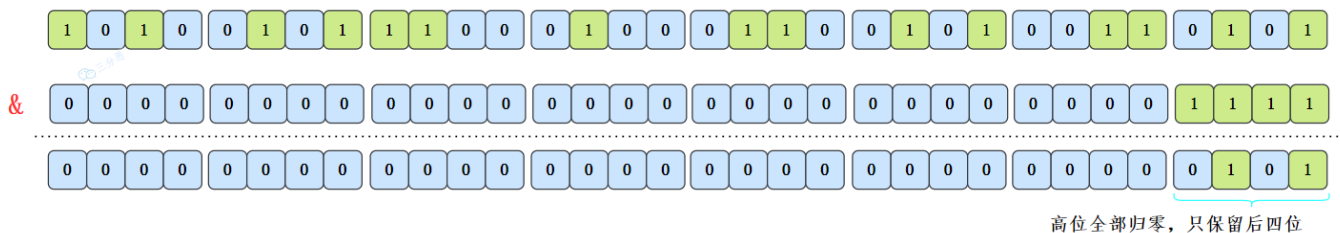
快速回答: 哈希表的索引是通过 $h \& (n-1)$ 计算的, n 是底层数组的容量; $n-1$ 和某个哈希值做 $\&$ 运算, 相当于截取了最低的四位。如果数组的容量很小, 只取 h 的低位很容易导致哈希冲突。

通过异或操作将 h 的高位引入低位, 可以增加哈希值的随机性, 从而减少哈希冲突。

解释一下。

```
static final int hash(Object key) { Complexity is 6 It's time to do something...
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

以初始长度 16 为例, $16-1=15$ 。2 进制表示是 0000 0000 0000 0000 0000 0000 0000 1111。只取最后 4 位相等于哈希值的高位都丢弃了。

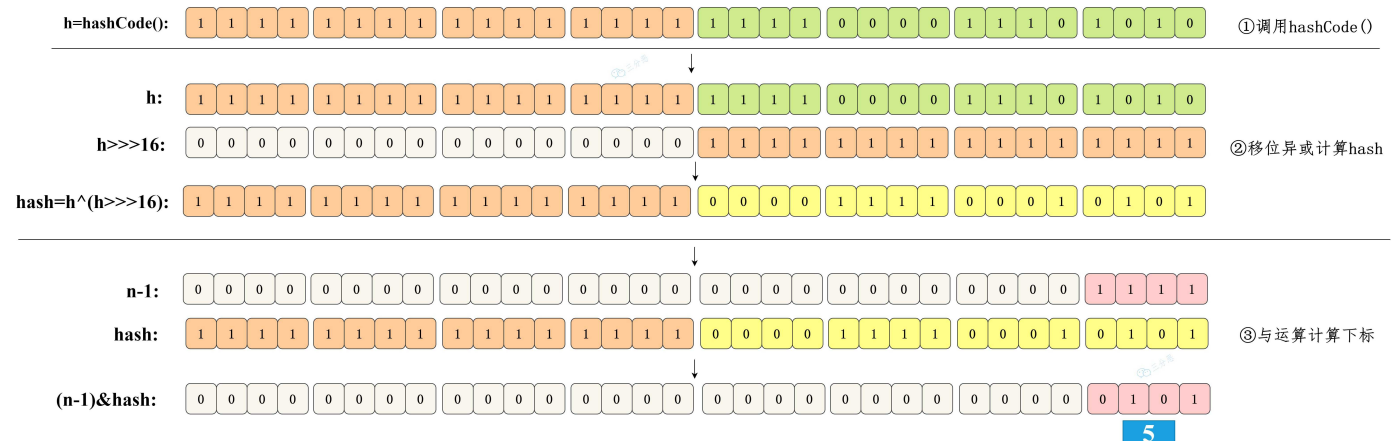


比如说 1111 1111 1111 1111 1111 1111 1111 1111, 取最后 4 位, 也就是 1111。

1110 1111 1111 1111 1111 1111 1111 1111, 取最后 4 位, 也是 1111。

不就发生哈希冲突了吗?

这时候 hash 函数 $(h = \text{key.hashCode()}) \wedge (h \ggg 16)$ 就派上用场了。



将哈希值无符号右移 16 位，意味着原哈希值的高 16 位被移到了低 16 位的位置。这样，原始哈希值的高 16 位和低 16 位就可以参与到最终用于索引计算的低位中。

选择 16 位是因为它是 32 位整数的一半，这样处理既考虑了高位的信息，又没有完全忽视低位原本的信息，从而达到了一种微妙的平衡状态。

举个例子（数组长度为 16）。

- 第一个键值对的键：h1 = 0001 0010 0011 0100 0101 0110 0111 1000
- 第二个键值对的键：h2 = 0001 0010 0011 0101 0101 0110 0111 1000

如果没有 hash 函数，直接取低 4 位，那么 h1 和 h2 的低 4 位都是 1000，也就是说两个键值对都会放在数组的第 8 个位置。

来看一下 hash 函数的处理过程。

①、对于第一个键 h1 的计算：

```
原始：0001 0010 0011 0100 0101 0110 0111 1000
右移：0000 0000 0000 0000 0001 0010 0011 0100
异或：-----
结果：0001 0010 0011 0100 0100 0100 0100 1100
```

②、对于第二个键 h2 的计算：

```
原始：0001 0010 0011 0101 0101 0110 0111 1000
右移：0000 0000 0000 0000 0001 0010 0011 0101
异或：-----
结果：0001 0010 0011 0101 0100 0100 0100 1101
```

通过上述计算，我们可以看到 h1 和 h2 经过 $h \oplus (h \gg 16)$ 操作后得到了不同的结果。

现在，考虑数组长度为 16 时（需要最低 4 位来确定索引）：

- 对于 h1 的最低 4 位是 1100（十进制中为 12）
- 对于 h2 的最低 4 位是 1101（十进制中为 13）

这样，h1 和 h2 就会被分别放在数组的第 12 个位置和第 13 个位置上，从而避免了哈希冲突。

1. [Java 面试指南（付费）](#) 收录的支付宝面经同学 2 春招技术一面面试原题：为什么要用高低做异或运算？为什么非得高低 16 位异或？

15.为什么 HashMap 的容量是 2 的幂次方？

是为了快速定位元素在底层数组中的下标。

HashMap 是通过 $hash \& (n-1)$ 来定位元素下标的，n 为数组的大小，也就是 HashMap 底层数组的容量。

数组长度-1 正好相当于一个“低位掩码”——掩码的低位最好全是 1，这样 & 运算才有意义，否则结果一定是 0。

2 幂次方刚好是偶数，偶数-1 是奇数，奇数的二进制最后一位是 1，也就保证了 $hash \& (length-1)$ 的最后一位可能为 0，也可能为 1（取决于 hash 的值），这样可以保证哈希值的均匀分布。

换句话说，& 操作的结果就是将哈希值的高位全部归零，只保留低位值。

a&b 的结果是：a、b 中对应位同时为 1，则结果为 1，否则为 0。例如 $5 \& 3 = 1$ ，5 的二进制是 0101，3 的二进制是 0011， $5 \& 3 = 0001 = 1$ 。

假设某哈希值的二进制为 10100101 11000100 00100101，用它来做 & 运算，我们来看一下结果。

已知 HashMap 的初始长度为 16， $16-1=15$ ，二进制是 00000000 00000000 00001111（高位用 0 来补齐）：

```

10100101 11000100 00100101
& 00000000 00000000 00001111
-----
00000000 00000000 00000101

```

因为 15 的高位全部是 0，所以 & 运算后的高位结果肯定也是 0，只剩下 4 个低位 0101，也就是十进制的 5。

这样，哈希值为 10100101 11000100 00100101 的键就会放在数组的第 5 个位置上。

对数组长度取模定位数组下标，这块有没有优化策略？

快速回答：HashMap 的策略是将取模运算 $\text{hash} \% \text{table.length}$ 优化为位运算 $\text{hash} \& (\text{length} - 1)$ 。

因为当数组的长度是 2 的 N 次幂时， $\text{hash} \& (\text{length} - 1) = \text{hash} \% \text{length}$ 。

比如说 $9 \% 4 = 1$ ，9 的二进制是 1001， $4 - 1 = 3$ ，3 的二进制是 0011， $9 \& 3 = 1001 \& 0011 = 0001 = 1$ 。

再比如说 $10 \% 4 = 2$ ，10 的二进制是 1010， $4 - 1 = 3$ ，3 的二进制是 0011， $10 \& 3 = 1010 \& 0011 = 0010 = 2$ 。

当数组的长度不是 2 的 n 次方时， $\text{hash} \% \text{length}$ 和 $\text{hash} \& (\text{length} - 1)$ 的结果就不一致了。

比如说 $7 \% 3 = 1$ ，7 的二进制是 0111， $3 - 1 = 2$ ，2 的二进制是 0010， $7 \& 2 = 0111 \& 0010 = 0010 = 2$ 。

从二进制角度来看， $\text{hash} / \text{length} = \text{hash} / 2^n = \text{hash} \gg n$ ，即把 hash 右移 n 位，此时得到了 $\text{hash} / 2^n$ 的商。

而被移调的部分，则是 $\text{hash} \% 2^n$ ，也就是余数。

2^n 的二进制形式为 1，后面跟着 n 个 0，那 $2^n - 1$ 的二进制则是 n 个 1。例如 $8 = 2^3$ ，二进制是 1000， $7 = 2^3 - 1$ ，二进制为 0111。

$\text{hash} \% \text{length}$ 的操作是求 hash 除以 2^n 的余数。在二进制中，这个操作的结果就是 hash 的二进制表示中最低 n 位的值。

因为在 2^n 取模的操作中，高于 2^n 表示位的所有数值对结果没有贡献，只有低于这个阈值的部分才决定余数。

比如说 26 的二进制是 11010，要计算 $26 \% 8$ ，8 是 2^3 ，所以我们关注的是 26 的二进制表示中最低 3 位：11010 的最低 3 位是 010。

010 对应于十进制中的 2， $26 \% 8$ 的结果是 2。

当执行 $\text{hash} \& (\text{length} - 1)$ 时，实际上是保留 hash 二进制表示的最低 n 位，其他高位都被清零。

举个例子，hash 为 14，n 为 3，也就是数组长度为 2^3 ，也就是 8。

```

1110 (hash = 14)
& 0111 (length - 1 = 7)
----
0110 (结果 = 6)

```

保留 14 的最低 3 位，高位被清零。

从此，两个运算 `hash % length` 和 `hash & (length - 1)` 有了完美的闭环。在计算机中，位运算的速度要远高于取余运算，因为计算机本质上就是二进制嘛。

说说什么是取模运算？

在 Java 中，通常使用 `%` 运算符来表示取余，用 `Math.floorMod()` 来表示取模。

当操作数都是正数的话，取模运算和取余运算的结果是一样的；只有操作数出现负数的情况下，结果才会不同。

取模运算的商向负无穷靠近；取余运算的商向 0 靠近。这是导致它们两个在处理有负数情况下，结果不同的根本原因。

当数组的长度是 2 的 n 次幂时，取模运算/取余运算可以用位运算来代替，效率更高，毕竟计算机本身只认二进制。

比如说，7 对 3 取余，和 7 对 3 取模，结果都是 1。因为两者都是基于除法运算的， $7 / 3$ 的商是 2，余数是 1。

对于 HashMap 来说，它需要通过 `hash % table.length` 来确定元素在数组中的位置。

比如说，数组长度是 3，hash 是 7，那么 $7 \% 3$ 的结果就是 1，也就是此时可以把元素放在下标为 1 的位置。

当 hash 是 8， $8 \% 3$ 的结果就是 2，也就是可以把元素放在下标为 2 的位置。

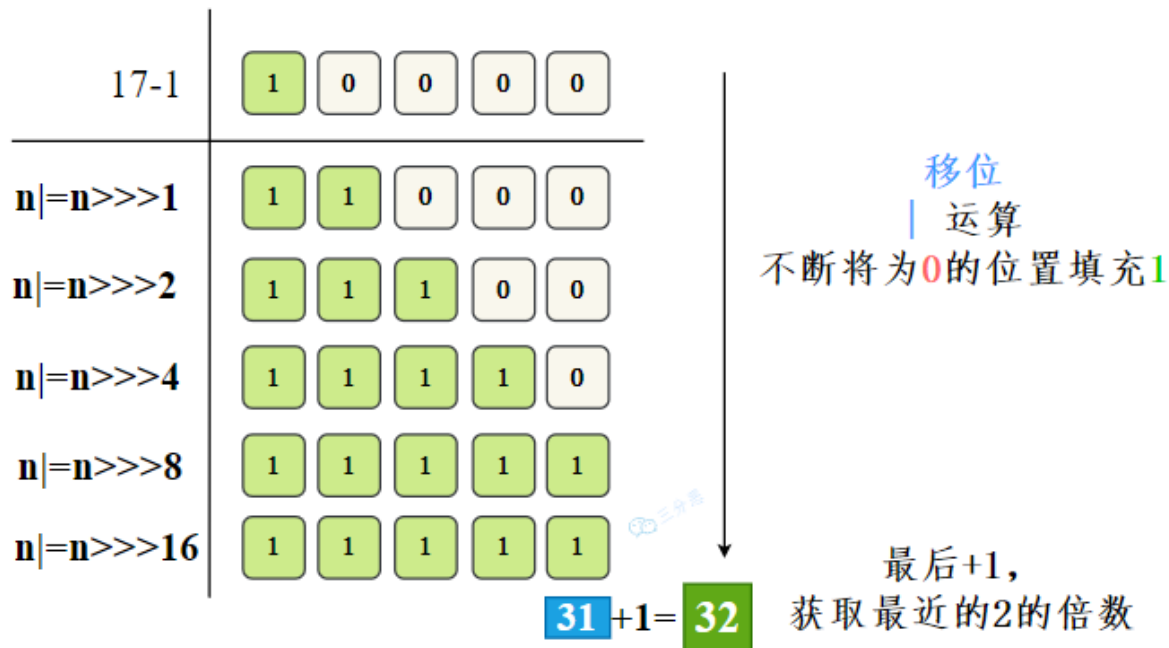
当 hash 是 9， $9 \% 3$ 的结果就是 0，也就是可以把元素放在下标为 0 的位置上。

是不是很奇妙，数组的大小为 3，刚好 3 个位置都利用上了。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程
2. [Java 面试指南（付费）](#) 收录的支付宝面经同学 2 春招技术一面面试原题：hashCode 对数组长度取模定位数组下标，这块有没有优化策略？

16.如果初始化 HashMap，传一个 17 的容量，它会怎么处理？

HashMap 会将容量调整到大于等于 17 的最小的 2 的幂次方，也就是 32。



这是因为哈希表的大小最好是 2 的 N 次幂，这样可以通过 $(n - 1) \& \text{hash}$ 高效计算出索引值。

解释一下。

在 HashMap 的初始化构造方法中，有这样一段代码：

```
public HashMap(int initialCapacity, float loadFactor) {
    ...
    this.loadFactor = loadFactor;
    this.threshold = tableSizeFor(initialCapacity);
}
```

阈值 threshold 会通过方法 `tableSizeFor()` 进行计算。

```
static final int tableSizeFor(int cap) {
    int n = cap - 1;
    n |= n >>> 1;
    n |= n >>> 2;
    n |= n >>> 4;
    n |= n >>> 8;
    n |= n >>> 16;
    return (n < 0) ? 1 : (n >= MAXIMUM_CAPACITY) ? MAXIMUM_CAPACITY : n + 1;
}
```

- ①、`int n = cap - 1;` 避免刚好是 2 的幂次方时，容量直接翻倍。
- ②、接下来通过不断右移 (`>>>`) 并与自身进行或运算 (`|=`)，将 n 的二进制表示中的所有低位设置为 1。
 - `n |= n >>> 1;` 将最高位的 1 扩展到下一位。
 - `n |= n >>> 2;` 扩展到后两位。
 - 依此类推，直到 `n |= n >>> 16;`，扩展到后十六位，这样从最高位的 1 到最低位，就都变成了 1。

③、如果 n 小于 0，说明 cap 是负数，直接返回 1。

如果 n 大于或等于 `MAXIMUM_CAPACITY`（通常是 2^{30} ），则返回 `MAXIMUM_CAPACITY`。

否则，返回 $n + 1$ ，这是因为 n 的所有低位都是 1，所以 $n + 1$ 就是大于 cap 的最小的 2 的幂次方。

初始化 HashMap 的时候需要传入容量吗？

如果预先知道 Map 将存储大量键值对，提前指定一个足够大的初始容量可以减少因扩容导致的重哈希操作。

因为每次扩容时，HashMap 需要将现有的元素插入到新的数组中，这个过程相对耗时，尤其是当 Map 中已有大量数据时。

当然了，过大的初始容量会浪费内存，特别是当实际存储的元素远少于初始容量时。如果不指定初始容量，HashMap 将使用默认的初始容量 16。

1. [Java 面试指南（付费）](#) 收录的奇安信面经同学 1 Java 技术一面面试原题：map 集合在使用时候一般都需要写容量值？为什么要写？扩容机制？

17.你还知道哪些哈希函数的构造方法呢？

①、**除留取余法**： $H(key) = key \% p (p \leq N)$ ，关键字除以一个不大于哈希表长度的正整数 p ，所得余数为地址，当然 HashMap 里进行了优化改造，效率更高，散列也更均衡。

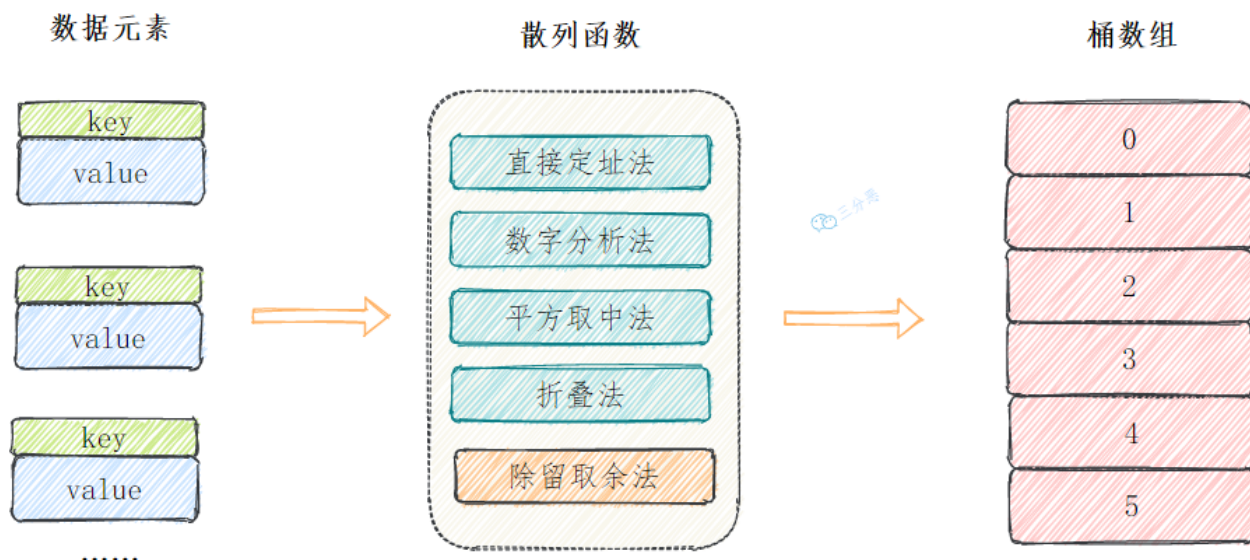
除此之外，还有这几种常见的哈希函数构造方法：

②、**直接定址法**：直接根据 `key` 来映射到对应的数组位置，例如 1232 放到下标 1232 的位置。

③、**数字分析法**：取 `key` 的某些数字（例如十位和百位）作为映射的位置

④、**平方取中法**：取 `key` 平方的中间几位作为映射的位置

⑤、将 `key` 分割成位数相同的几段，然后把它们的叠加和作为映射的位置。



18.解决哈希冲突有哪些方法？

简版回答：我知道的有 3 种，再哈希法、开放地址法和拉链法。

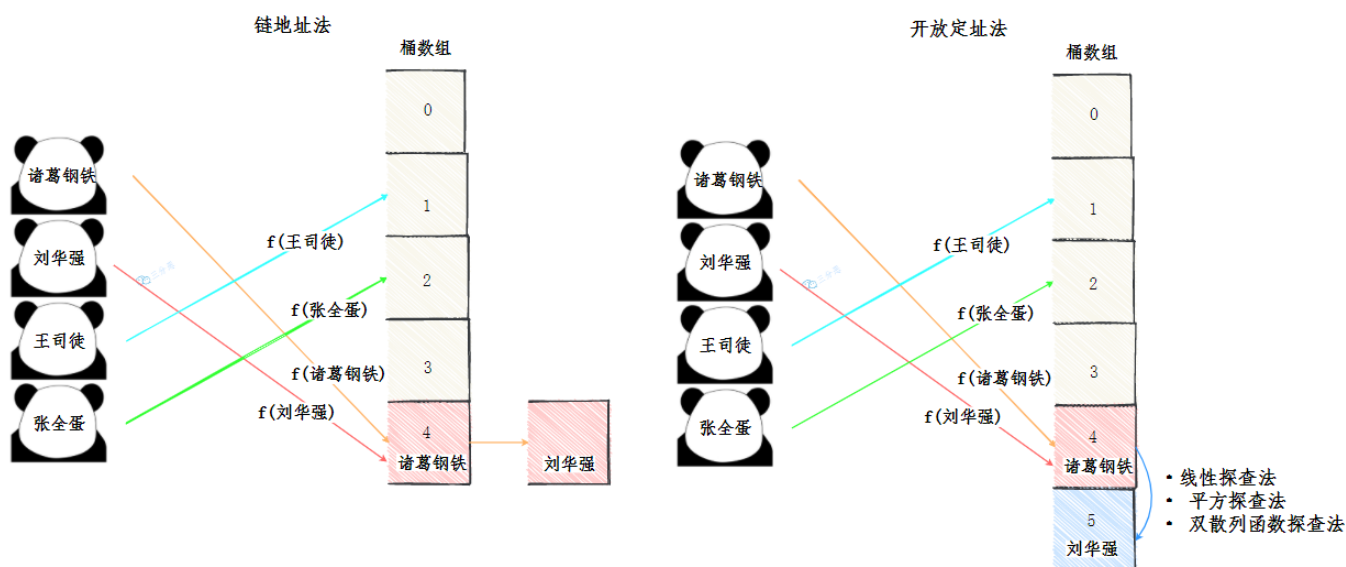
什么是再哈希法？

准备两套哈希算法，当发生哈希冲突的时候，使用另外一种哈希算法，直到找到空槽为止。对哈希算法的设计要求比较高。

什么是开放地址法？

遇到哈希冲突的时候，就去寻找下一个空的槽。有 3 种方法：

- 线性探测：从冲突的位置开始，依次往后找，直到找到空槽。
- 二次探测：从冲突的位置 x 开始，第一次增加 1^2 个位置，第二次增加 2^2 ，直到找到空槽。
- 双重哈希：和再哈希法类似，准备多个哈希函数，发生冲突的时候，使用另外一个哈希函数。



什么是拉链法？

也就是链地址法，当发生哈希冲突的时候，使用链表将冲突的元素串起来。HashMap 采用的正是拉链法。

怎么判断 key 相等呢？

依赖于 key 的 `equals()` 方法和 `hashCode()` 方法。

```
if (e.hash == hash &&
    ((k = e.key) == key || (key != null && key.equals(k))))
```

①、`hashCode()`：使用 key 的 `hashCode()` 方法计算 key 的哈希码。

②、`equals()`：当两个 key 的哈希码相同时，HashMap 还会调用 key 的 `equals()` 方法进行精确比较。只有当 `equals()` 方法返回 `true` 时，两个 key 才被认为是完全相同的。

如果两个 key 的引用指向了同一个对象，那么它们的 `hashCode()` 和 `equals()` 方法都会返回 `true`，所以在 `equals` 判断之前可以先使用 `==` 运算符判断一次。

1. [Java 面试指南 \(付费\)](#) 收录的支付宝面经同学 2 春招技术一面面试原题：HashMap 怎么解决冲突？怎么判断 key 相等？

19.为什么 HashMap 链表转红黑树的阈值为 8 呢？

树化发生在 table 数组的长度大于 64，且链表的长度大于 8 的时候。

为什么是 8 呢？源码的注释也给出了答案。

```
*
* Because TreeNodes are about twice the size of regular nodes, we
* use them only when bins contain enough nodes to warrant use
* (see TREEIFY_THRESHOLD). And when they become too small (due to
* removal or resizing) they are converted back to plain bins. In
* usages with well-distributed user hashCodes, tree bins are
* rarely used. Ideally, under random hashCodes, the frequency of
* nodes in bins follows a Poisson distribution
* (http://en.wikipedia.org/wiki/Poisson\_distribution) with a
* parameter of about 0.5 on average for the default resizing
* threshold of 0.75, although with a large variance because of
* resizing granularity. Ignoring variance, the expected
* occurrences of list size k are (exp(-0.5) * pow(0.5, k) /
* factorial(k)). The first values are:
*
* 0: 0.60653066
* 1: 0.30326533
* 2: 0.07581633
* 3: 0.01263606
* 4: 0.00157952
* 5: 0.00015795
* 6: 0.00001316
* 7: 0.00000094
* 8: 0.00000006
* more: less than 1 in ten million
```

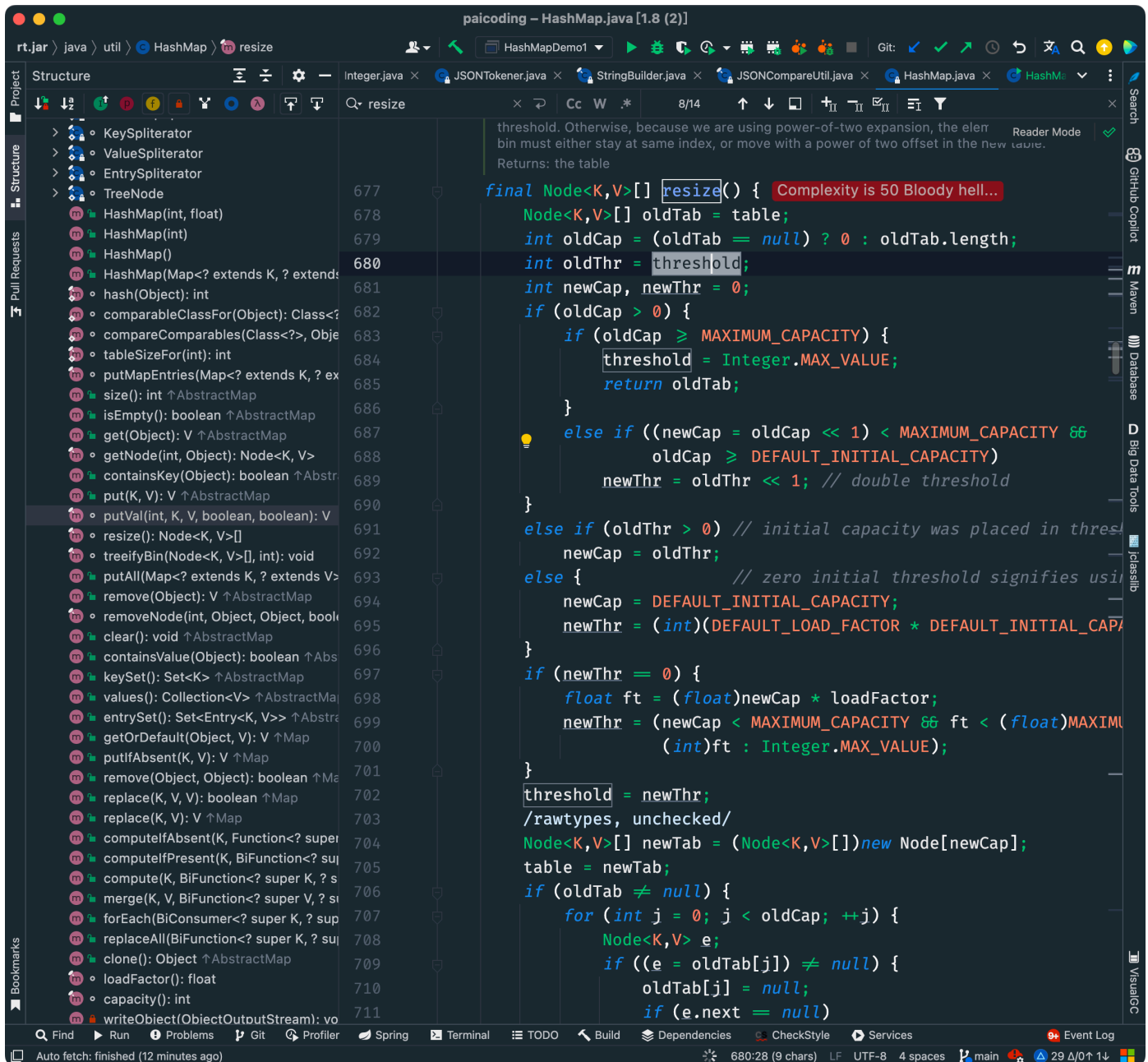
红黑树节点的大小大概是普通节点大小的两倍，所以转红黑树，牺牲了空间换时间，更多的是一种兜底的策略，保证极端情况下的查找效率。

阈值为什么要选 8 呢？和统计学有关。理想情况下，使用随机哈希码，链表里的节点符合泊松分布，出现节点个数的概率是递减的，节点个数为 8 的情况，发生概率仅为 0.00000006。

至于红黑树转回链表的阈值为什么是 6，而不是 8？是因为如果这个阈值也设置成 8，假如发生碰撞，节点增减刚好在 8 附近，会发生链表和红黑树的不断转换，导致资源浪费。

20.HashMap扩容发生在什么时候呢？

当键值对数量超过阈值，也就是容量 * 负载因子时。



默认的负载因子是多少？

0.75。

初始容量是多少？

16。

1 左移 4 位，`0000 0001 → 0001 0000`，也就是 2 的 4 次方。

```
static final int DEFAULT_INITIAL_CAPACITY = 1 << 4; // aka 16
```

为什么使用 `1 << 4` 而不是直接写 16？

写 `1 << 4` 主要是为了强调这个值是 2 的幂次方，而不是一个完全随机的选择。

无论 HashMap 是否扩容，其底层的数组长度都应该是 2 的幂次方，因为这样可以通过位运算快速计算出元素的索引。

为什么选择 0.75 作为 HashMap 的默认负载因子呢？

这是一个经验值。如果设置得太低，如 0.5，会浪费空间；如果设置得太高，如 0.9，会增加哈希冲突。

```

69      * <p>As a general rule, the default load factor (.75) offers a good
70      * tradeoff between time and space costs. Higher values decrease the
71      * space overhead but increase the lookup cost (reflected in most of
72      * the operations of the <tt>HashMap</tt> class, including
73      * <tt>get</tt> and <tt>put</tt>). The expected number of entries in
74      * the map and its load factor should be taken into account when
75      * setting its initial capacity, so as to minimize the number of
76      * rehash operations. If the initial capacity is greater than the
77      * maximum number of entries divided by the load factor, no rehash
78      * operations will ever occur.
79      *
80      *
81      *
82      *
83      *

```

作为一般规则，默认负载系数（.75）在时间和空间成本之间提供了很好的权衡。较高的值会减少空间开销，但会增加查找成本（反映在 HashMap 类的大多数操作中，包括 get 和 put）。在设置其初始容量时，应考虑 map 中的预期条目数及其负载因子，以最大限度地减少 rehash 操作的数量。如果初始容量大于最大条目数除以负载因子，则不会发生重新哈希操作。

0.75 是 JDK 作者经过大量验证后得出的最优解，能够最大限度减少 rehash 的次数。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程

memo：2025 年 1 月 7 日第二版优化到此。

21. 🌟 HashMap 的扩容机制了解吗？

扩容时，HashMap 会创建一个新的数组，其容量是原来的两倍。然后遍历旧哈希表中的元素，将其重新分配到新的哈希表中。

如果当前桶中只有一个元素，那么直接通过键的哈希值与数组大小取模锁定新的索引位置：`e.hash & (newCap - 1)`。

如果当前桶是红黑树，那么会调用 `split()` 方法分裂树节点，以保证树的平衡。

如果当前桶是链表，会通过旧键的哈希值与旧的数组大小取模 `(e.hash & oldCap) == 0` 来作为判断条件，如果条件为真，元素保留在原索引的位置；否则元素移动到原索引 + 旧数组大小的位置。

JDK 7 扩容的时候有什么问题？

JDK 7 在扩容的时候使用头插法来重新插入链表节点，这样会导致链表无法保持原有的顺序。

详细解释一下。

JDK 7 是通过哈希值与数组大小-1 进行与运算确定元素下标的。

```

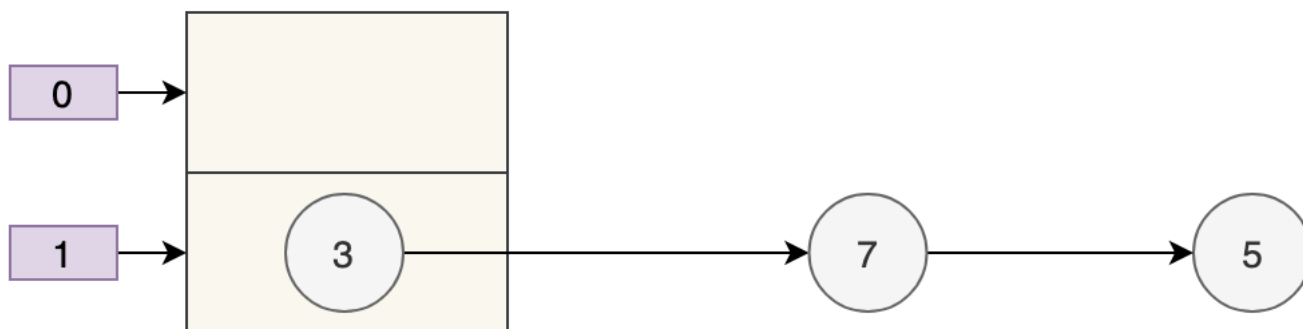
static int indexFor(int h, int length) {
    return h & (length-1);
}

```


我们来假设：

- 数组 table 的长度为 2
- 键的哈希值为 3、7、5

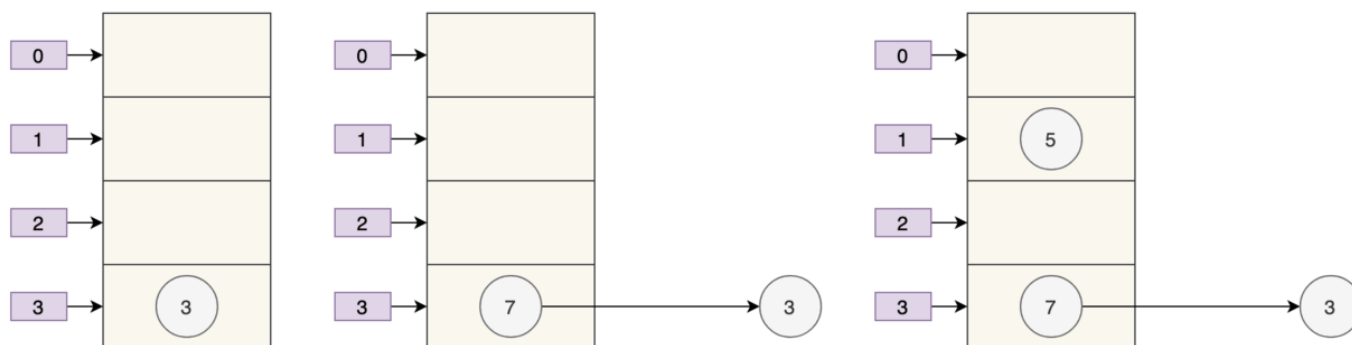
取模运算后，键发生了哈希冲突，它们都需要放到 `table[1]` 的桶上。那么扩容前就是这个样子：



假设负载因子 loadFactor 为 1，也就是当元素的个数大于 table 的长度时进行扩容。

扩容后的数组容量为 4。

- key 3 取模 ($3\%4$) 后是 3，放在 `table[3]` 上。
- key 7 取模 ($7\%4$) 后是 3，放在 `table[3]` 上的链表头部。
- key 5 取模 ($5\%4$) 后是 1，放在 `table[1]` 上。



可以看到，由于 JDK 采用的是头插法，7 跑到 3 的前面了，原来的顺序是 3、7、5，7 在 3 的后面。

```

for (Entry<K,V> e : oldTable) {
    while (null != e) {
        Entry<K,V> next = e.next;
        int i = indexFor(e.hash, newCapacity);
        e.next = newTable[i];
        newTable[i] = e;
        e = next;
    }
}

```

最好的情况就是，扩容后的 7 还在 3 的后面，保持原来的顺序。

JDK 8 是怎么解决这个问题的?

JDK 8 改用了尾插法，并且当 `(e.hash & oldCap) == 0` 时，元素保留在原索引的位置；否则元素移动到原索引 + 旧数组大小的位置。

```
Node<K,V> loHead = null, loTail = null;
Node<K,V> hiHead = null, hiTail = null;
Node<K,V> next;
do {
    next = e.next;
    if ((e.hash & oldCap) == 0) {
        if (loTail == null)
            loHead = e;
        else
            loTail.next = e;
        loTail = e;
    }
    else {
        if (hiTail == null)
            hiHead = e;
        else
            hiTail.next = e;
        hiTail = e;
    }
} while ((e = next) != null);
if (loHead != null)
    newTab[j] = loHead;
if (hiHead != null)
    newTab[j + oldCap] = hiHead;
```

由于扩容时，数组长度会翻倍，例如：16 → 32， 因此，新数组的索引范围是原索引范围的两倍。

原索引 `index = (n - 1) & hash`，扩容后的新索引就是 `index = (2n - 1) & hash`。

也就是说，如果 `(e.hash & oldCap) == 0`，元素在新数组中的位置与旧位置相同；否则，元素在新数组中的位置是旧位置 + 旧数组大小。

假设扩容前的数组长度为 16 ($n-1$ 也就是二进制的 0000 1111， $1 \times \{2^0\} + 1 \times \{2^1\} + 1 \times \{2^2\} + 1 \times \{2^3\} = 1 + 2 + 4 + 8 = 15$)，key1 为 5（二进制为 0000 0101），key2 为 21（二进制为 0001 0101）。

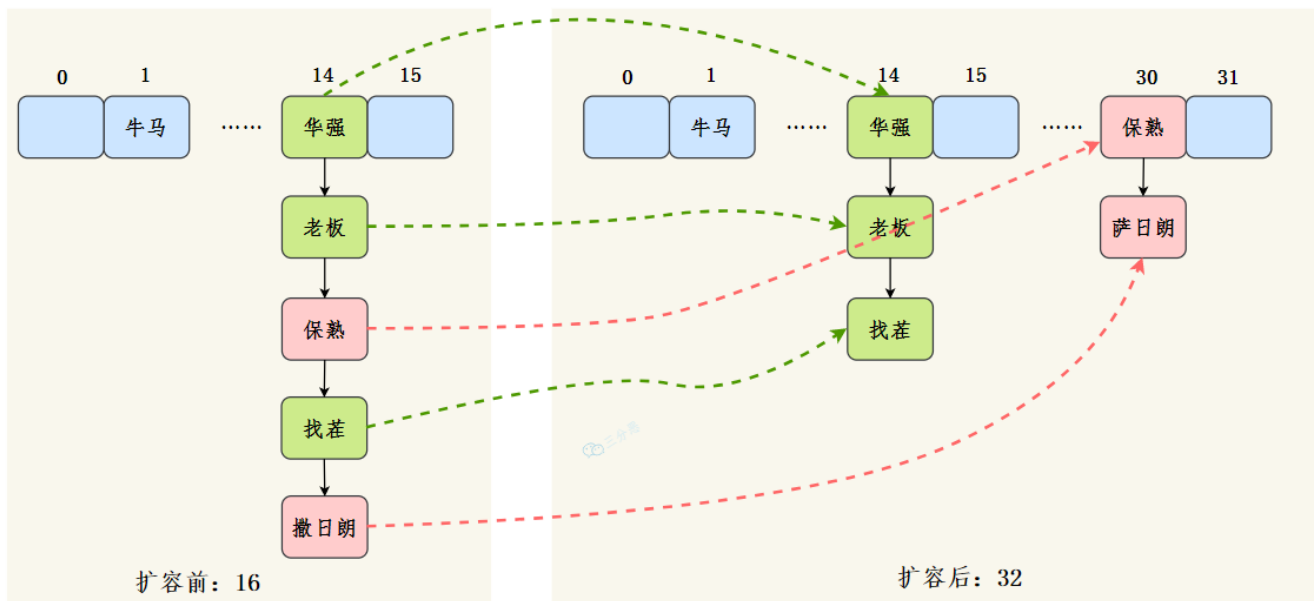
- key1 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5；
- key2 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5。
- 此时哈希冲突了，用拉链法来解决哈希冲突。

现在，HashMap 进行了扩容，容量为原来的 2 倍，也就是 32 ($n-1$ 也就是二进制的 0001 1111， $1 \times \{2^0\} + 1 \times \{2^1\} + 1 \times \{2^2\} + 1 \times \{2^3\} + 1 \times \{2^4\} = 1 + 2 + 4 + 8 + 16 = 31$)。

- key1 和 $n-1$ 做 & 运算后为 0000 0101，也就是 5；
- key2 和 $n-1$ 做 & 运算后为 0001 0101，也就是 $21 = 5 + 16$ ，就是数组扩容前的位置+原数组的长度。

$$0101=5 \xrightarrow[16 \Rightarrow 2 \times 16]{\text{resize}} \begin{matrix} 0 & 0101 & =5 & \text{原位置} \\ 1 & 0101 & =21 = 5 + 16 & \text{原位置} + \text{oldCap} \end{matrix}$$

这样可以避免重新计算所有元素的哈希值，只需检查高位的某一位，就可以快速确定新位置。



扩容的时候每个节点都要进行位运算吗？

不需要。HashMap 会通过 `(e.hash & oldCap)` 来判断节点是否需要移动，0 的话保留原索引；1 才需要移动到新索引（原索引 + oldCap）。

这样就避免了 hashCode 的重新计算，大大提升了扩容的性能。

所以，哪怕有几十万条数据，可能只有一半的数据才需要移动到新位置。另外，位运算的计算速度非常快，因此，尽管扩容操作涉及到遍历整个哈希表并对每个节点进行判断，但这部分操作的计算成本是相对较低的。

1. [Java 面试指南（付费）](#) 收录的小米春招同学 K 一面面试原题：为什么是 2 次幂 到什么时候开始扩容 扩容机制流程
2. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：说说 HashMap 的扩容机制，1.8 扩容具体实现
3. [Java 面试指南（付费）](#) 收录的奇安信面经同学 1 Java 技术一面面试原题：map 集合在使用时候一般都需要写容量值？为什么要写？扩容机制？
4. [Java 面试指南（付费）](#) 收录的百度面经同学 1 文心一言 25 实习 Java 后端面试原题：hashmap 的底层实现原理、put()方法实现流程、扩容机制？

22.JDK 8 对 HashMap 做了哪些优化呢？

①、底层数据结构由数组 + 链表改成了数组 + 链表或红黑树的结构。

如果多个键映射到了同一个哈希值，链表会变得很长，在最坏的情况下，当所有的键都映射到同一个桶中时，性能会退化到 $O(n)$ ，而红黑树的时间复杂度是 $O(\log n)$ 。

②、链表的插入方式由头插法改为了尾插法。头插法在扩容后容易改变原来链表的顺序。

③、扩容的时机由插入时判断改为插入后判断，这样可以避免在每次插入时都进行不必要的扩容检查，因为有可能插入后仍然不需要扩容。

```
void addEntry(int hash, K key, V value, int bucketIndex)
    if ((size >= threshold) && (null != table[bucketIndex]))
        resize(2 * table.length);
    hash = (null != key) ? hash(key) : 0;
    bucketIndex = indexFor(hash, table.length);
}

createEntry(hash, key, value, bucketIndex);
}
```

```
final V putVal(int hash, K key, V value, boolean onlyIfAbsent,
               boolean evict) {
    Node<K, V>[] tab; Node<K, V> p; int n, i;
    if ((tab = table) == null || (n = tab.length) == 0)
        n = (tab = resize()).length;
    if ((p = tab[i = (n - 1) & hash]) == null)
        tab[i] = newNode(hash, key, value, next: null);
    else {...}
    ++modCount;
    if (++size > threshold)
        resize();
    afterNodeInsertion(evict);
    return null;
}
```

④、哈希扰动算法也进行了优化。JDK 7 是通过多次移位和异或运算来实现的。

```
final int hash(Object k) { Complexity is 11 You must be kidding
    int h = hashSeed;
    if (0 != h && k instanceof String) {
        return sun.misc.Hashing.stringHash32((String) k);
    }

    h ^= k.hashCode();

    // This function ensures that hashCodes that differ only by
    // constant multiples at each bit position have a bounded
    // number of collisions (approximately 8 at default load factor).
    h ^= (h >> 20) ^ (h >> 12);
    return h ^ (h >> 7) ^ (h >> 4);
}
```

JDK 8 让 hash 值的高 16 位和低 16 位进行了异或运算，让高位的信息也能参与到低位的计算中，这样可以极大程度上减少哈希碰撞。

```
static final int hash(Object key) { Complexity is 6 It's time to do something...
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16);
}
```

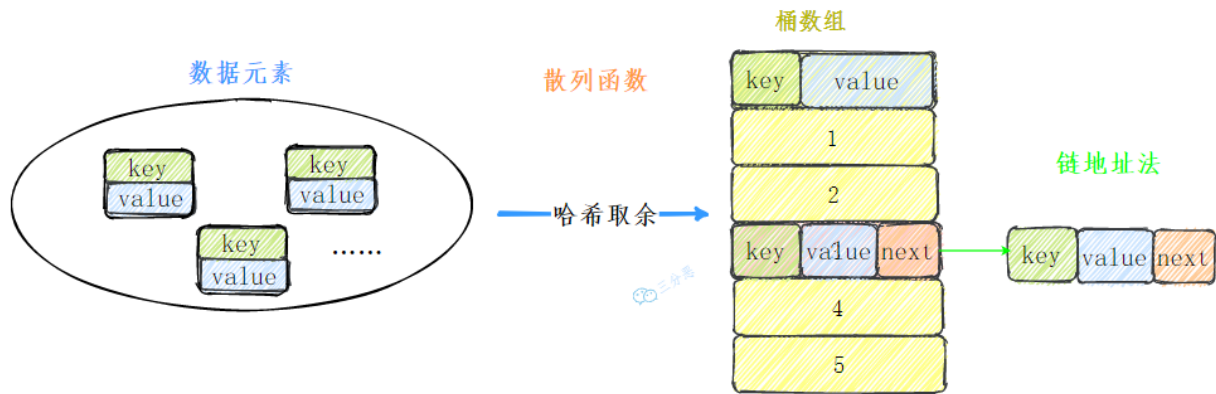
1. [Java 面试指南 \(付费\)](#) 收录的美团同学 2 优选物流调度技术 2 面面试原题：HashMap 的内部结构，1.7 和 1.8 的区别，有什么改进

23.你能自己设计实现一个 HashMap 吗？

这道题快手常考。红黑树版咱们多半是写不出来的，但是数组+链表版还是问题不大，详细可见：[手写 HashMap, 快手面试官直呼内行!](#)。

可以，我先说一下整体的设计思路：

- 第一步，实现一个 hash 函数，对键的 hashCode 进行扰动
- 第二步，实现一个拉链法的方法来解决哈希冲突
- 第三步，扩容后，重新计算哈希值，将元素放到新的数组中



完整代码：

```
/**
 * @Author 三分恶
 * @Date 2021/11/21
 * @Description 自己实现HashMap
 */
public class ThirdHashMap<K, V> {

    /**
     * 节点类
     */
    @param <K>
    @param <V>
    */
    class Node<K, V> {
        //键值对
        private K key;
        private V value;

        //链表，后继
        private Node<K, V> next;

        public Node(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public Node(K key, V value, Node<K, V> next) {
            this.key = key;
            this.value = value;
            this.next = next;
        }
    }

    //默认容量
```

```

final int DEFAULT_CAPACITY = 16;
//负载因子
final float LOAD_FACTOR = 0.75f;
//HashMap的大小
private int size;
//桶数组
Node<K, V>[] buckets;

/**
 * 无参构造器，设置桶数组默认容量
 */
public ThirdHashMap() {
    buckets = new Node[DEFAULT_CAPACITY];
    size = 0;
}

/**
 * 有参构造器，指定桶数组容量
 *
 * @param capacity
 */
public ThirdHashMap(int capacity) {
    buckets = new Node[capacity];
    size = 0;
}

/**
 * 哈希函数，获取地址
 *
 * @param key
 * @return
 */
private int getIndex(K key, int length) {
    //获取hash code
    int hashCode = key.hashCode();
    //和桶数组长度取余
    int index = hashCode % length;
    return Math.abs(index);
}

/**
 * put方法
 *
 * @param key
 * @param value
 * @return
 */
public void put(K key, V value) {
    //判断是否需要进行扩容
    if (size >= buckets.length * LOAD_FACTOR) resize();
    putVal(key, value, buckets);
}

/**
 * 将元素存入指定的node数组
 *

```

```

    * @param key
    * @param value
    * @param table
    */
    private void putVal(K key, V value, Node<K, V>[] table) {
        //获取位置
        int index = getIndex(key, table.length);
        Node node = table[index];
        //插入的位置为空
        if (node == null) {
            table[index] = new Node<>(key, value);
            size++;
            return;
        }
        //插入位置不为空, 说明发生冲突, 使用链地址法, 遍历链表
        while (node != null) {
            //如果key相同, 就覆盖掉
            if ((node.key.hashCode() == key.hashCode())
                && (node.key == key || node.key.equals(key))) {
                node.value = value;
                return;
            }
            node = node.next;
        }
        //当前key不在链表中, 插入链表头部
        Node newNode = new Node(key, value, table[index]);
        table[index] = newNode;
        size++;
    }

    /**
     * 扩容
     */
    private void resize() {
        //创建一个两倍容量的桶数组
        Node<K, V>[] newBuckets = new Node[buckets.length * 2];
        //将当前元素重新散列到新的桶数组
        rehash(newBuckets);
        buckets = newBuckets;
    }

    /**
     * 重新散列当前元素
     *
     * @param newBuckets
     */
    private void rehash(Node<K, V>[] newBuckets) {
        //map大小重新计算
        size = 0;
        //将旧的桶数组的元素全部刷到新的桶数组里
        for (int i = 0; i < buckets.length; i++) {
            //为空, 跳过
            if (buckets[i] == null) {
                continue;
            }
            Node<K, V> node = buckets[i];
            while (node != null) {
                rehashNode(node, newBuckets);
                node = node.next;
            }
        }
    }

```

```

        Node<K, V> node = buckets[i];
        while (node != null) {
            //将元素放入新数组
            putVal(node.key, node.value, newBuckets);
            node = node.next;
        }
    }

    /**
     * 获取元素
     *
     * @param key
     * @return
     */
    public V get(K key) {
        //获取key对应的地址
        int index = getIndex(key, buckets.length);
        if (buckets[index] == null) return null;
        Node<K, V> node = buckets[index];
        //查找链表
        while (node != null) {
            if ((node.key.hashCode() == key.hashCode())
                && (node.key == key || node.key.equals(key))) {
                return node.value;
            }
            node = node.next;
        }
        return null;
    }

    /**
     * 返回HashMap大小
     *
     * @return
     */
    public int size() {
        return size;
    }
}

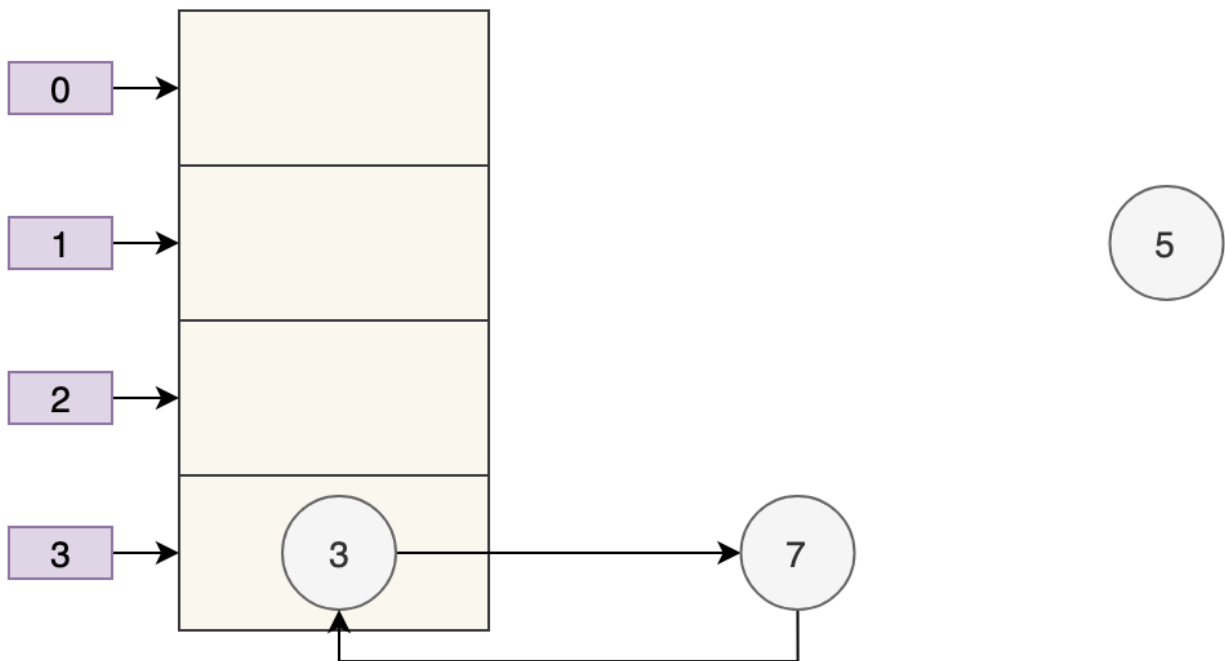
```

24. 🌟 HashMap 是线程安全的吗?

推荐阅读: [HashMap 详解](#)

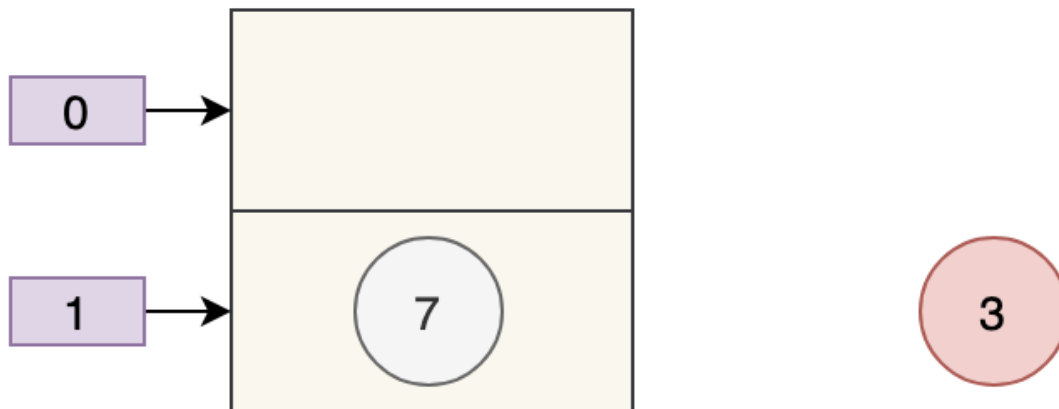
HashMap 不是线程安全的, 主要有以下几个问题:

①、多线程下扩容会死循环。JDK7 中的 HashMap 使用的是头插法来处理链表, 在多线程环境下扩容会出现环形链表, 造成死循环。

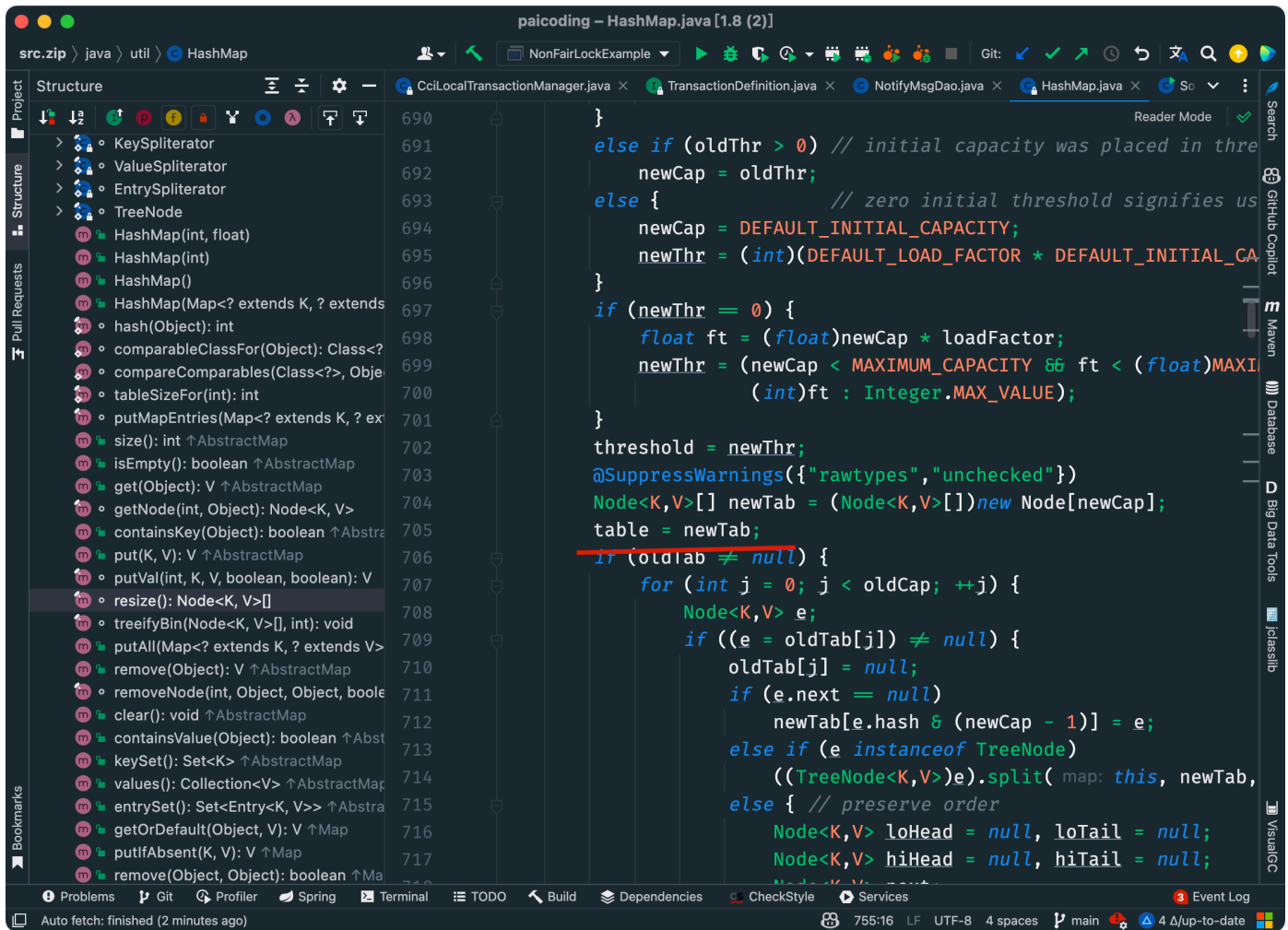


不过，JDK 8 时通过尾插法修复了这个问题，扩容时会保持链表原来的顺序。

②、多线程在进行 put 元素的时候，可能会导致元素丢失。因为计算出来的位置可能会被其他线程覆盖掉，比如说一个县城 put 3 的时候，另外一个线程 put 了 7，就把 3 给弄丢了。



③、put 和 get 并发时，可能导致 get 为 null。线程 1 执行 put 时，因为元素个数超出阈值而扩容，线程 2 此时执行 get，就有可能出现这个问题。



因为线程 1 执行完 `table = newTab` 之后，线程 2 中的 `table` 已经发生了改变，比如说索引 3 的键值对移动到了索引 7 的位置，此时线程 2 去 `get` 索引 3 的元素就 `get` 不到了。

1. [Java 面试指南（付费）](#) 收录的华为 OD 原题：HashMap 是线程安全的吗？
2. [Java 面试指南（付费）](#) 收录的华为面经同学 8 技术二面面试原题：HashMap 是线程安全的吗？
3. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 9 飞书后端技术一面试原题：HashMap 为什么不安全，如何改进，以及 ConcurrentHashMap
4. [Java 面试指南（付费）](#) 收录的腾讯云智面经同学 16 一面试原题：HashMap 的底层实现，它为什么是线程不安全的？
5. [Java 面试指南（付费）](#) 收录的京东同学 4 云实习面试原题：hashmap是会死锁的，你知道吗
6. [Java 面试指南（付费）](#) 收录的比亚迪面经同学 12 Java 技术面试原题：map的同步和非同步
7. [Java 面试指南（付费）](#) 收录的 OPPO 面经同学 1 面试原题：为什么HashMap不是线程安全的？

25. 🌟 怎么解决 HashMap 线程不安全的问题呢？

在早期的 JDK 版本中，可以用 Hashtable 来保证线程安全。Hashtable 在方法上加了 [synchronized 关键字](#)。

从 Java 2 平台 v1.2 开始，该类被修改为实现接口 `Map`，使其成为 **Java 集合框架** 的成员。与新的集合实现不同，`Hashtable` 它是同步的。如果不需要线程安全实现，建议使用 `HashMap` 代替 `Hashtable`。如果需要线程安全的高并发实现，则建议使用 `java.util.concurrent.ConcurrentHashMap` 代替 `Hashtable`。

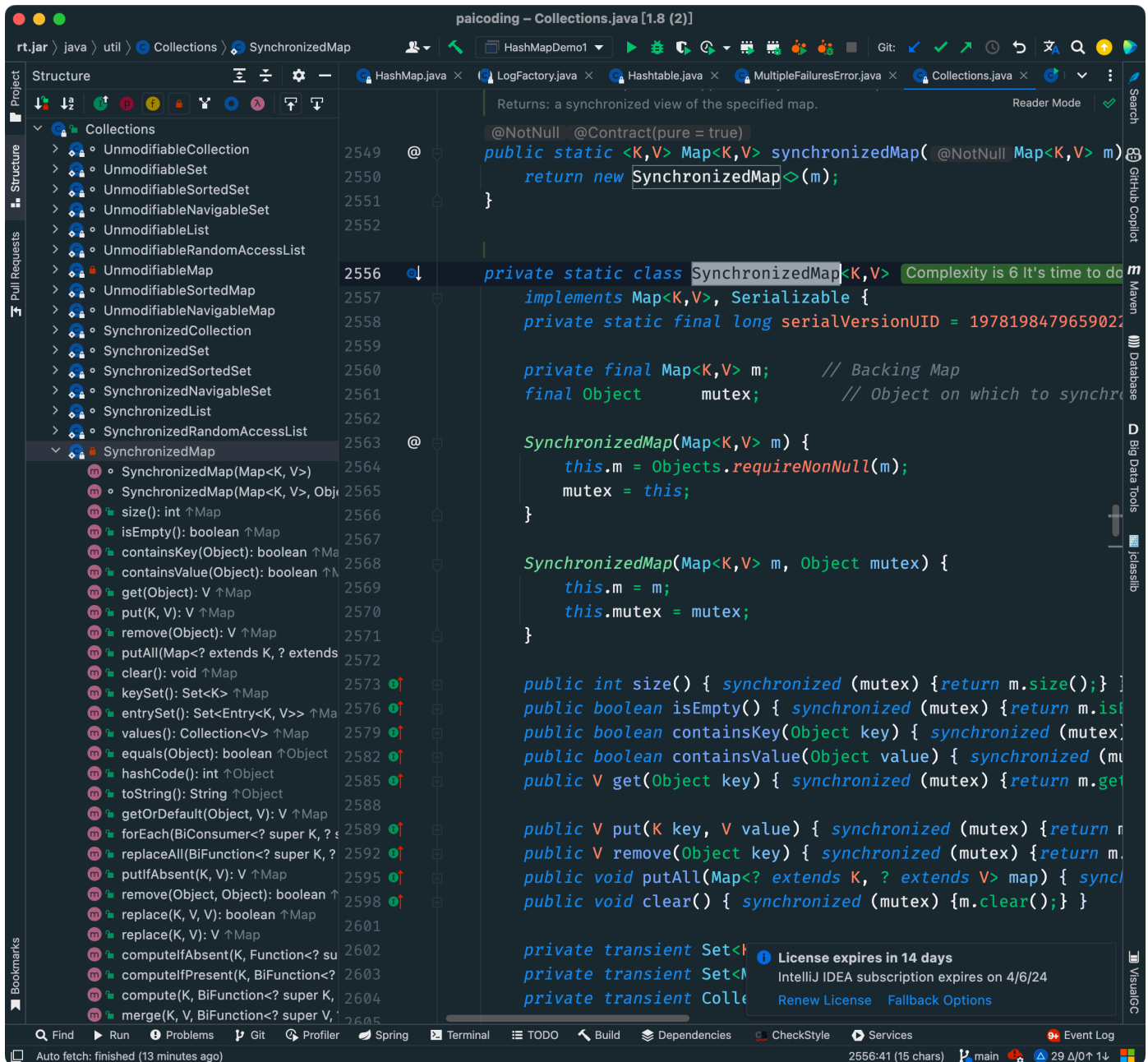
因为： JDK1.0

另请参见： `Object.equals(Object)`,
`Object.hashCode()`,
`rehash()`,
`Collection`,
`Map`,
`HashMap`,
`TreeMap`

作者： 亚瑟·范霍夫、乔什·布洛赫、尼尔·格夫特

```
public class Hashtable<K,V> Complexity is 28 Bloody hell...  
    extends Dictionary<K,V>  
    implements Map<K,V>, Cloneable, java.io.Serializable {
```

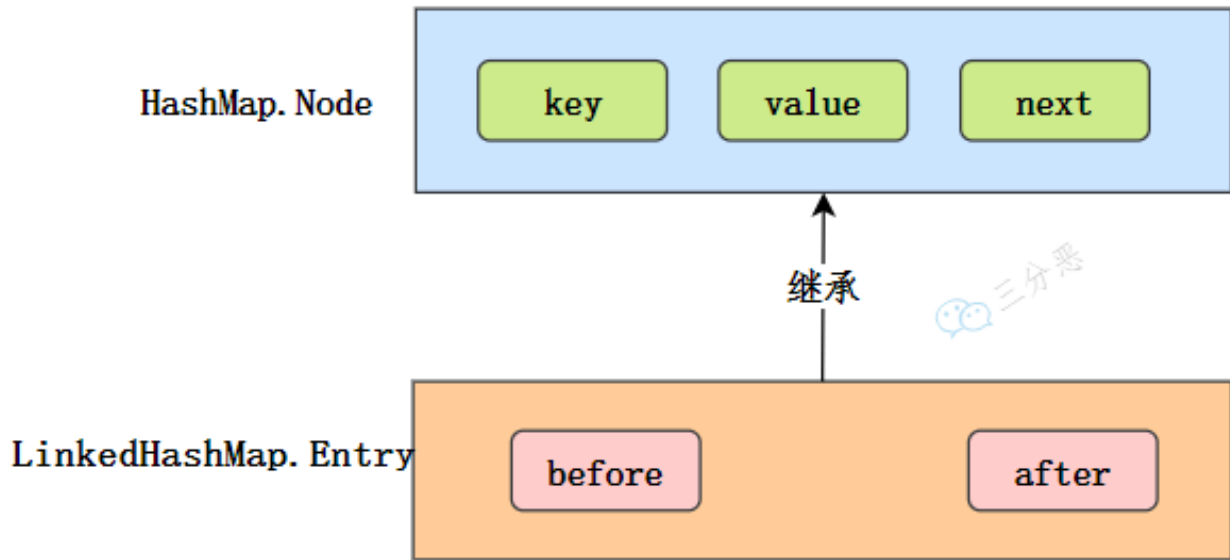
另外，可以通过 `Collections.synchronizedMap` 方法返回一个线程安全的 `Map`，内部是通过 `synchronized` 对象锁来保证线程安全的，比在方法上直接加 `synchronized` 关键字更轻量级。



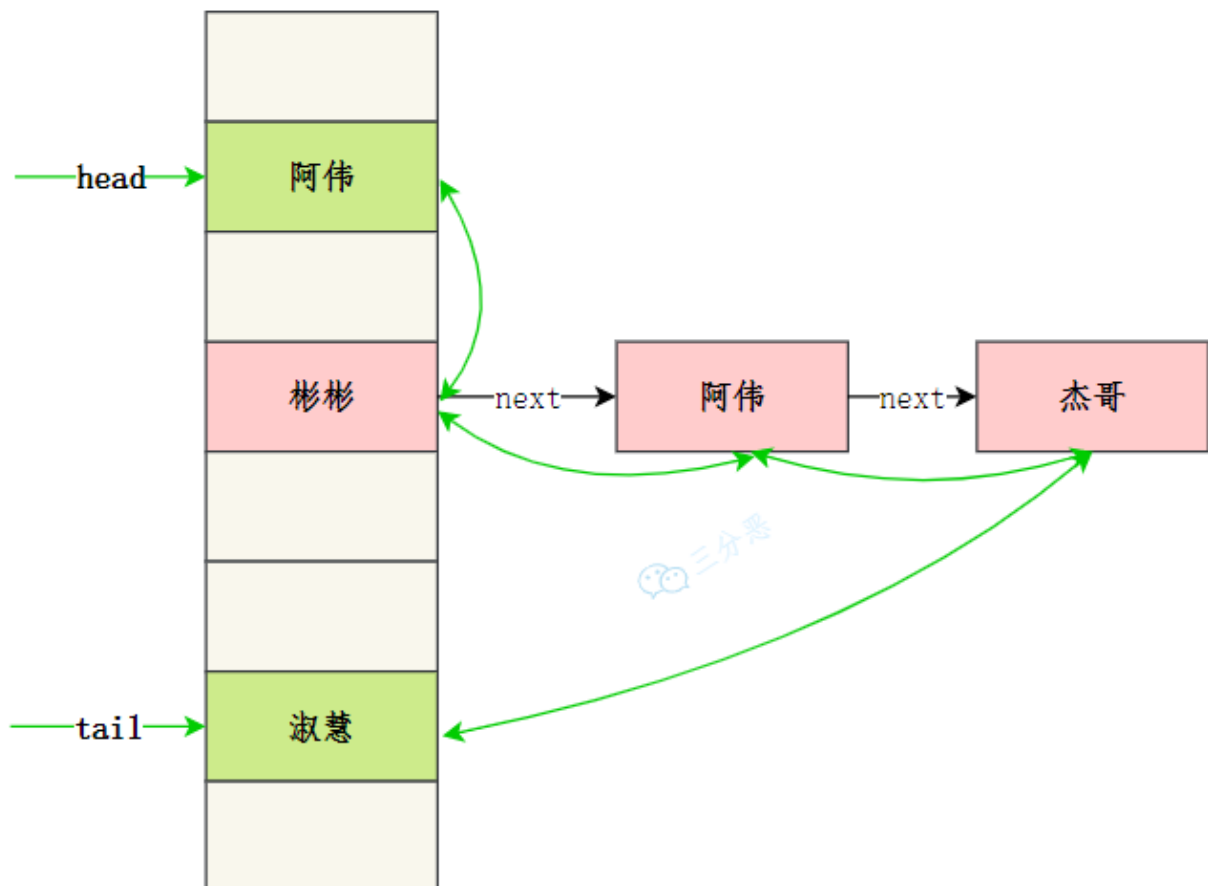
更优雅的方案是使用并发工具包下的 `ConcurrentHashMap`，使用了 `CAS` + `synchronized` 关键字来保证线程安全。

27.讲讲 LinkedHashMap 怎么实现有序的？

LinkedHashMap 在 HashMap 的基础上维护了一个双向链表，通过 before 和 after 标识前置节点和后置节点。

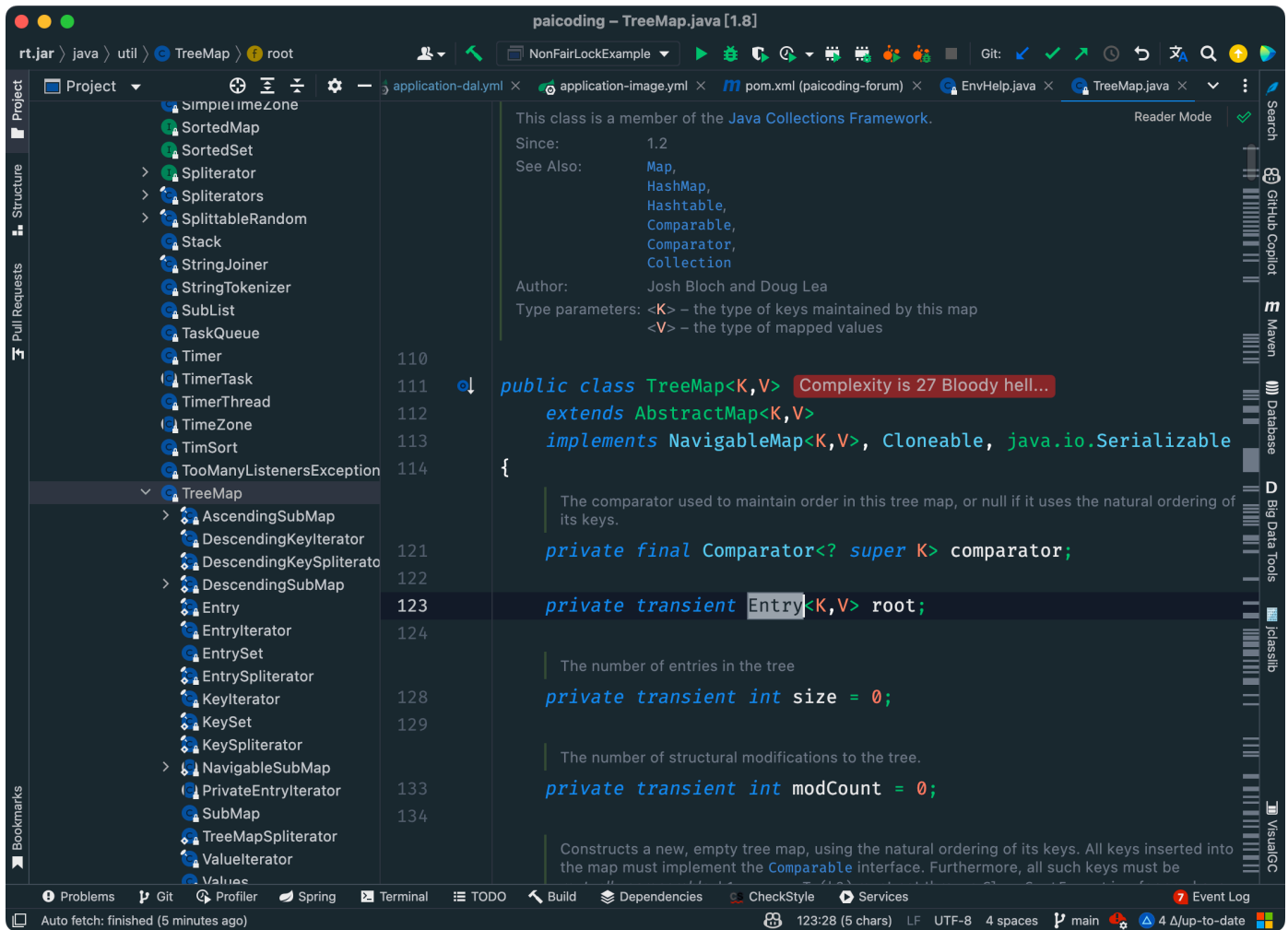


从而实现插入的顺序或访问顺序。

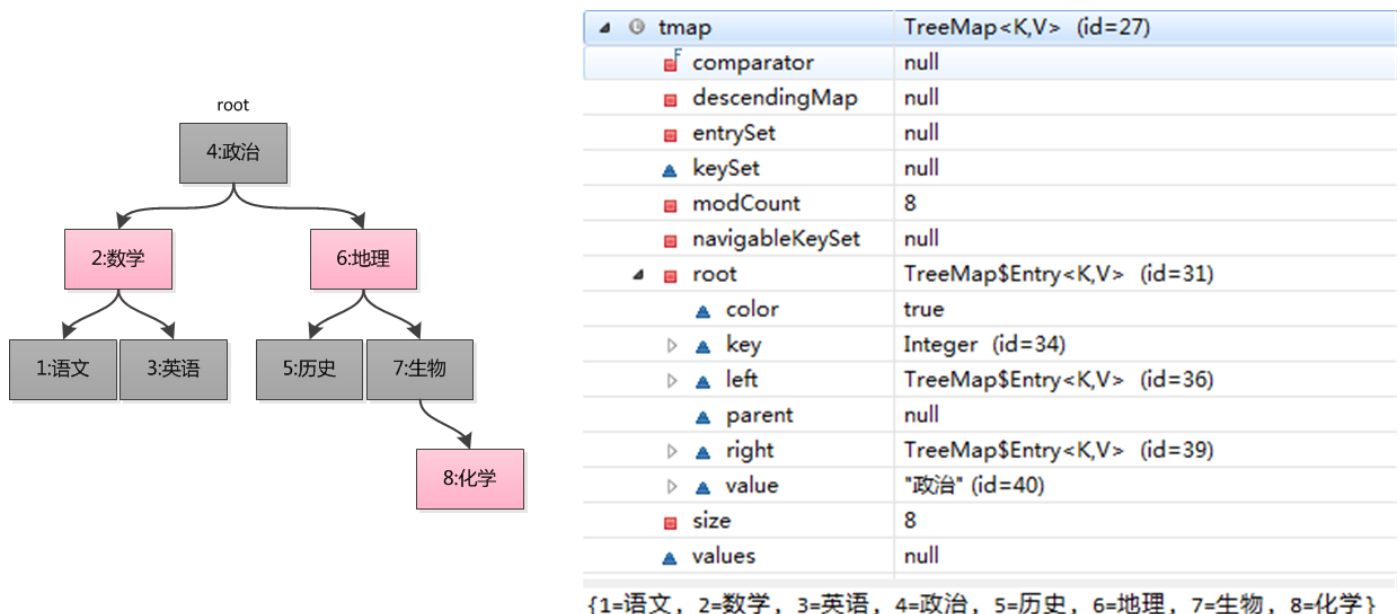


28.讲讲 TreeMap 怎么实现有序的？

`TreeMap` 通过 `key` 的比较器来决定元素的顺序，如果没有指定比较器，那么 `key` 必须实现 [Comparable 接口](#)。



TreeMap 的底层是红黑树，红黑树是一种自平衡的二叉查找树，每个节点都大于其左子树中的任何节点，小于其右子节点树种的任何节点。



插入或者删除元素时通过旋转和染色来保持树的平衡。

查找的时候从根节点开始，利用二叉查找树的特点，逐步向左子树或者右子树递归查找，直到找到目标元素。

29.TreeMap 和 HashMap 的区别

①、HashMap 是基于数组+链表+红黑树实现的，put 元素的时候会先计算 key 的哈希值，然后通过哈希值计算出元素在数组中的存放下标，然后将元素插入到指定的位置，如果发生哈希冲突，会使用链表来解决，如果链表长度大于 8，会转换为红黑树。

②、TreeMap 是基于红黑树实现的，put 元素的时候会先判断根节点是否为空，如果为空，直接插入到根节点，如果不为空，会通过 key 的比较器来判断元素应该插入到左子树还是右子树。

在没有发生哈希冲突的情况下，HashMap 的查找效率是 $O(1)$ 。适用于查找操作比较频繁的场景。

TreeMap 的查找效率是 $O(\log n)$ 。并且保证了元素的顺序，因此适用于需要大量范围查找或者有序遍历的场景。

1. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：知道哪些集合，讲讲 HashMap 和 TreeMap 的区别，讲讲两者应用场景的区别
2. [Java 面试指南（付费）](#) 收录的快手同学 2 一面面试原题：HashMap 和 TreeMap 区别

Set

30.讲讲 HashSet 的底层实现？

HashSet 是由 HashMap 实现的，只不过值由一个固定的 Object 对象填充，而键用于操作。

```
public class HashSet<E>
    extends AbstractSet<E>
    implements Set<E>, Cloneable, java.io.Serializable
{
    static final long serialVersionUID = -5024744406713321676L;
    private transient HashMap<E, Object> map;
    // Dummy value to associate with an Object in the backing Map
    private static final Object PRESENT = new Object();
    // .....
}
```

实际开发中，HashSet 并不常用，比如，如果我们需要按照顺序存储一组元素，那么 ArrayList 和 LinkedList 更适合；如果我们需要存储键值对并根据键进行查找，那么 HashMap 可能更适合。

HashSet 主要用于去重，比如，我们需要统计一篇文章中有多少个不重复的单词，就可以使用 HashSet 来实现。

```
// 创建一个 HashSet 对象
HashSet<String> set = new HashSet<>();

// 添加元素
set.add("沉默");
set.add("王二");
set.add("陈清扬");
set.add("沉默");

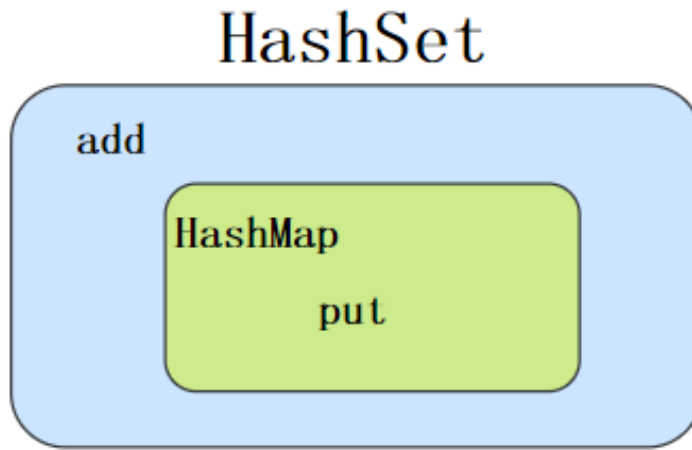
// 输出 HashSet 的元素个数
System.out.println("HashSet size: " + set.size()); // output: 3

// 遍历 HashSet
for (String s : set) {
```



```
System.out.println(s);  
}
```

HashSet 会自动去重，因为它用 HashMap 实现的，HashMap 的键是唯一的，相同键会覆盖掉原来的键，于是第二次 add 一个相同键的元素会直接覆盖掉第一次的键。



HashSet 和 ArrayList 的区别

- ArrayList 是基于动态数组实现的，HashSet 是基于 HashMap 实现的。
- ArrayList 允许重复元素和 null 值，可以有多个相同的元素；HashSet 保证每个元素唯一，不允许重复元素，基于元素的 hashCode 和 equals 方法来确定元素的唯一性。
- ArrayList 保持元素的插入顺序，可以通过索引访问元素；HashSet 不保证元素的顺序，元素的存储顺序依赖于哈希算法，并且可能随着元素的添加或删除而改变。

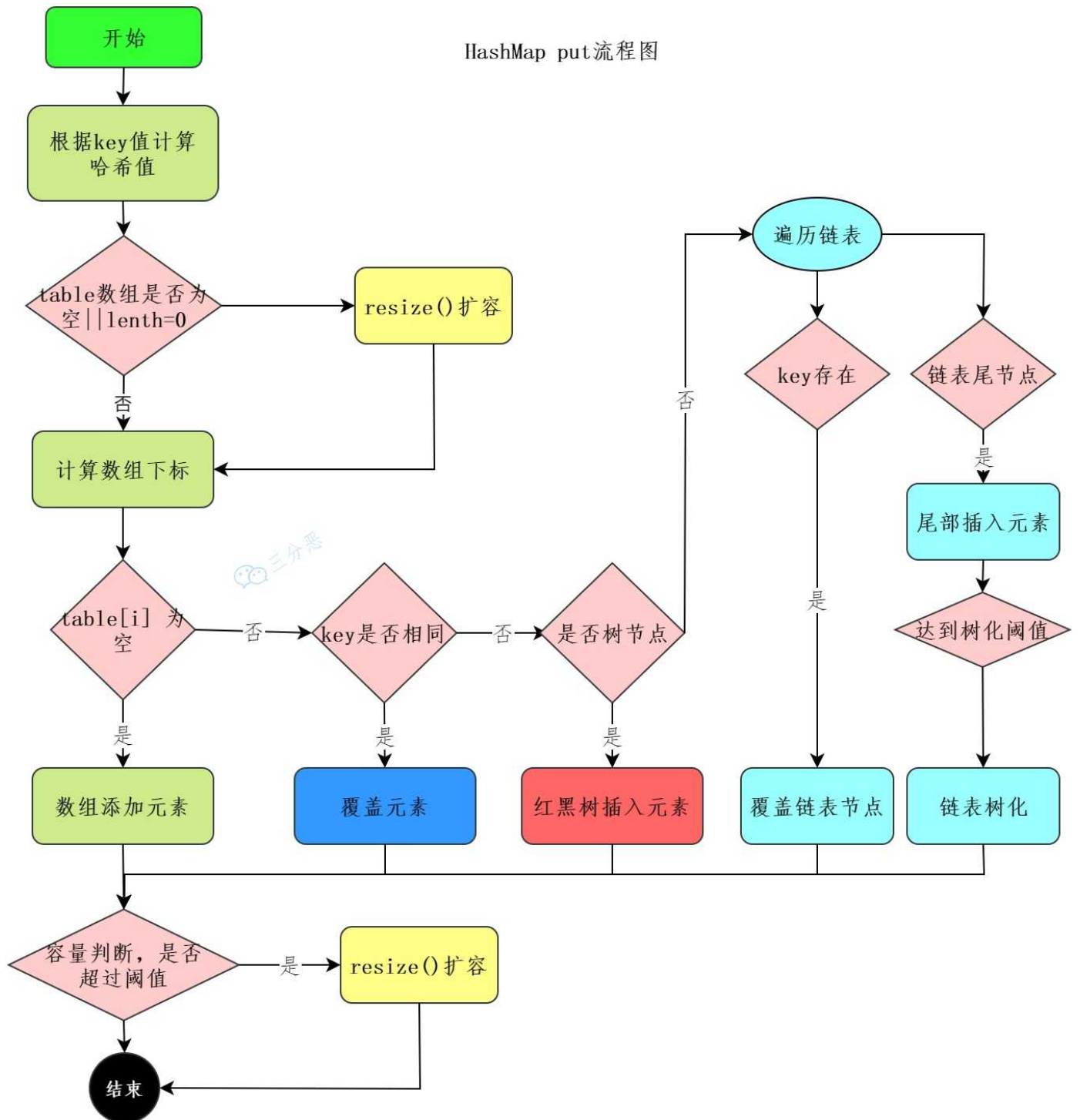
HashSet 怎么判断元素重复，重复了是否 put

HashSet 的 add 方法是通过调用 HashMap 的 put 方法实现的：

```
public boolean add(E e) {  
    return map.put(e, PRESENT) == null;  
}
```

所以 HashSet 判断元素重复的逻辑底层依然是 HashMap 的底层逻辑：

HashMap put流程图



HashMap 在插入元素时，通常需要三步：

第一步，通过 hash 方法计算 key 的哈希值。

```

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
  
```

第二步，数组进行第一次扩容。

```
if ((tab = table) == null || (n = tab.length) == 0)
    n = (tab = resize()).length;
```

第三步，根据哈希值计算 key 在数组中的下标，如果对应下标正好没有存放数据，则直接插入。

```
if ((p = tab[i = (n - 1) & hash]) == null)
    tab[i] = newNode(hash, key, value, null);
```

如果对应下标已经有数据了，就需要判断是否为相同的 key，是则覆盖 value，否则需要判断是否为树节点，是则向树中插入节点，否则向链表中插入数据。

```
else {
    Node<K,V> e; K k;
    if (p.hash == hash &&
        ((k = p.key) == key || (key != null && key.equals(k))))
        e = p;
    else if (p instanceof TreeNode)
        e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);
    else {
        for (int binCount = 0; ; ++binCount) {
            if ((e = p.next) == null) {
                p.next = newNode(hash, key, value, null);
                if (binCount >= TREEIFY_THRESHOLD - 1) // -1 for 1st
                    treeifyBin(tab, hash);
                break;
            }
            if (e.hash == hash &&
                ((k = e.key) == key || (key != null && key.equals(k))))
                break;
            p = e;
        }
    }
}
```

也就是说，HashSet 通过元素的哈希值来判断元素是否重复，如果重复了，会覆盖原来的值。

```
if (e != null) { // existing mapping for key
    V oldValue = e.value;
    if (!onlyIfAbsent || oldValue == null)
        e.value = value;
    afterNodeAccess(e);
    return oldValue;
}
```

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：HashSet 和 ArrayList 的区别
2. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：HashSet 怎么判断元素重复，重复了是否 put

说一点心里话。

网上的八股其实不少，这样可以给大家提供更多的选择。

面渣逆袭第二版是在星球嘉宾三分恶的初版基础上，加入了二哥自己的思考，加入了 1000 多份真实面经之后的结果，并且从 24 届到 25 届，帮助了很多小伙伴。未来的 26、27 届，也将因此受益，从而拿到心仪的 offer。

能帮助到大家，我很欣慰，并且在重制面渣逆袭的过程中，我也成长了很多，很多薄弱的基础环节都得到了加强。



花舞洛伊人 2 进阶牛

11-19 21:46 内蒙古农业大学 Java 发布于内蒙古

八股文看谁的

如题，javaguide，二哥面渣，小林code，看谁的🤔



银行究极大孝子 6 大橘已定 🐼 潜力领航者

南京理工大学 Java

全是广告哥，我觉得二哥面渣/小林就足够了，你不一定要只看一家，一个八股问题，看其他博客，集百家之长

👍 36 💬 回复 📄 分享 发布于 09-03 14:44 江苏



1589494222 3 出师牛 🐼 潜力领航者 楼主 回复

中肯的

👍 1 💬 回复 发布于 09-03 20:55 北京 来自Android客户端



流连~ 5 飞黄腾达

深圳技术大学 Java

二哥的面渣逆袭，还有掘金竹子爱熊猫的专栏，个人感觉还是不错的

👍 32 💬 回复 📄 分享 发布于 09-01 14:46 广东 来自Android客户端



陕西 4天前



双非硕测开开了17×15拒了🤔二哥八股文无敌



沉默王二 作者 4天前



感谢认可，八股是不是吊打面试官😊



； 陕西 4天前



回复 沉默王二：包吊打的👍

很多时候，我觉得自己是一个佛系的人，不愿意和别人争个高低，也不愿意去刻意宣传自己的作品。

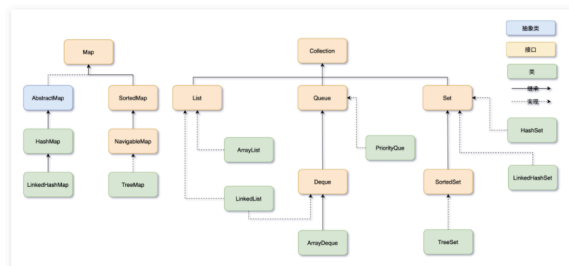
我喜欢静待花开。

如果你觉得面渣逆袭还不错，可以告诉学弟学妹们有这样一份免费的学习资料。

我还会继续优化，也不确定第三版什么时候会来，但我会尽力。

愿大家都有一个光明的未来。

这次仍然是三个版本，亮白、暗黑和 epub 版本。给大家展示其中一个 epub 版本吧，有些小伙伴很急需这个版本，所以也满足大家了。



集合框架可以分为两条大的支线：

①、第一条支线 Collection，主要由 List、Set、Queue 组成：

- List 代表有序、可重复的集合，典型代表就是封装了动态数组的 [ArrayList](#) 和封装了链表的 [LinkedList](#)；
- Set 代表无序、不可重复的集合，典型代表就是 [HashSet](#) 和 [TreeSet](#)；
- Queue 代表队列，典型代表就是双端队列 [ArrayDeque](#)，以及优先级队列 [PriorityQueue](#)。

②、第二条支线 Map，代表键值对的集合，典型代表就是 [HashMap](#)。

另外一个回答版本：

①、Collection 接口：最基本的集合框架表示方式，提供了添加、删除、清空等基本操作，它主要有三个子接口：

- List：一个有序的集合，可以包含重复的元素。实现类包括 [ArrayList](#)、[LinkedList](#) 等。
- Set：一个不包含重复元素的集合。实现类包括 [HashSet](#)、[LinkedHashSet](#)、[TreeSet](#) 等。
- Queue：一个用于保持元素队列的集合。实现类包括 [PriorityQueue](#)、[ArrayDeque](#) 等。

②、Map 接口：表示键值对的集合，一个键映射到一个值。键不能重复，每个键只能对应一个值。Map 接口的实现类包括 [HashMap](#)、[LinkedHashMap](#)、[TreeMap](#) 等。

集合框架有哪几个常用工具类？

集合框架位于 `java.util` 包下，提供了两个常用的工具类：

- [Collections](#)：提供了一些对集合进行排序、二分查找、同步的静态方法。
- [Arrays](#)：提供了一些对数组进行排序、打印、和 List 进行转换的静态方法。

由于 PDF 没办法自我更新，所以需要最新版的小伙伴，可以微信搜【沉默王二】，或者扫描/长按识别下面的二维码，关注二哥的公众号，回复【222】即可拉取最新版本。



当然了，请允许我的一点点私心，那就是星球的 PDF 版本会比公众号早一个月时间，毕竟星球用户都付费过了，我有必要让他们先享受到一点点福利。相信大家也都能理解，毕竟在线版是免费的，CDN、服务器、域名、OSS 等等都是需要成本的。

更别说我付出的时间和精力了，大家觉得有帮助还请给个口碑，让你身边的同事、同学都能受益到。

The image is a screenshot of a WeChat interface. On the left, there's a chat window with a contact named 'wanganpan'. The chat history shows a message about a salary of 60*13 and another about a Java PDF. Below that, there's a message about a Java concurrency book and links to 'javabetter.cn'. At the bottom left, there's a green bubble with the number '222'. On the right, there's a list of files shared by 'wanganpan'. The files include various PDFs and EPUBs related to Java, JVM, and system administration, with dates ranging from 2024 to 2025. The file '面渣逆袭 JVM篇 V2.0.pdf' is highlighted in blue. The interface includes standard WeChat icons for back, forward, search, and a list of contacts.

我把二哥的 Java 进阶之路、JVM 进阶之路、并发编程进阶之路，以及所有面渣逆袭的版本都放进来了，涵盖 Java 基础、Java 集合、Java 并发、JVM、Spring、MyBatis、计算机网络、操作系统、MySQL、Redis、RocketMQ、分布式、微服务、设计模式、Linux 等 16 个大的主题，共有 40 多万字，2000+张手绘图，可以说是诚意满满。

图文详解 29 道 Java 集合框架面试高频题，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接](#)，作者：三分恶，戳[原文链接](#)。

没有什么使我停留——除了目的，纵然岸旁有玫瑰、有绿荫、有宁静的港湾，我是不系之舟。

系列内容：

- [面渣逆袭 Java SE 篇](#) 
- [面渣逆袭 Java 集合框架篇](#) 
- [面渣逆袭 Java 并发编程篇](#) 

- [面渣逆袭 JVM 篇](#) 🍷
- [面渣逆袭 Spring 篇](#) 🍷
- [面渣逆袭 Redis 篇](#) 🍷
- [面渣逆袭 MyBatis 篇](#) 🍷
- [面渣逆袭 MySQL 篇](#) 🍷
- [面渣逆袭操作系统篇](#) 🍷
- [面渣逆袭计算机网络篇](#) 🍷
- [面渣逆袭 RocketMQ 篇](#) 🍷
- [面渣逆袭分布式篇](#) 🍷
- [面渣逆袭微服务篇](#) 🍷
- [面渣逆袭设计模式篇](#) 🍷
- [面渣逆袭 Linux 篇](#) 🍷